LOW LEVEL DESIGN · CASE STUDY

Rate Limiter System Design

Study notes that build a thread-safe, per-tier rate limiter in six phases: requirements, UML, the fixed window, sliding window log and token bucket algorithms, the Factory pattern, per-user locking with `ConcurrentHashMap.compute`, and a complete walk-through of the code flow. Every phase has its own folder of runnable Java code.

1 **Users, Tiers & Limits**
entities · enums · config

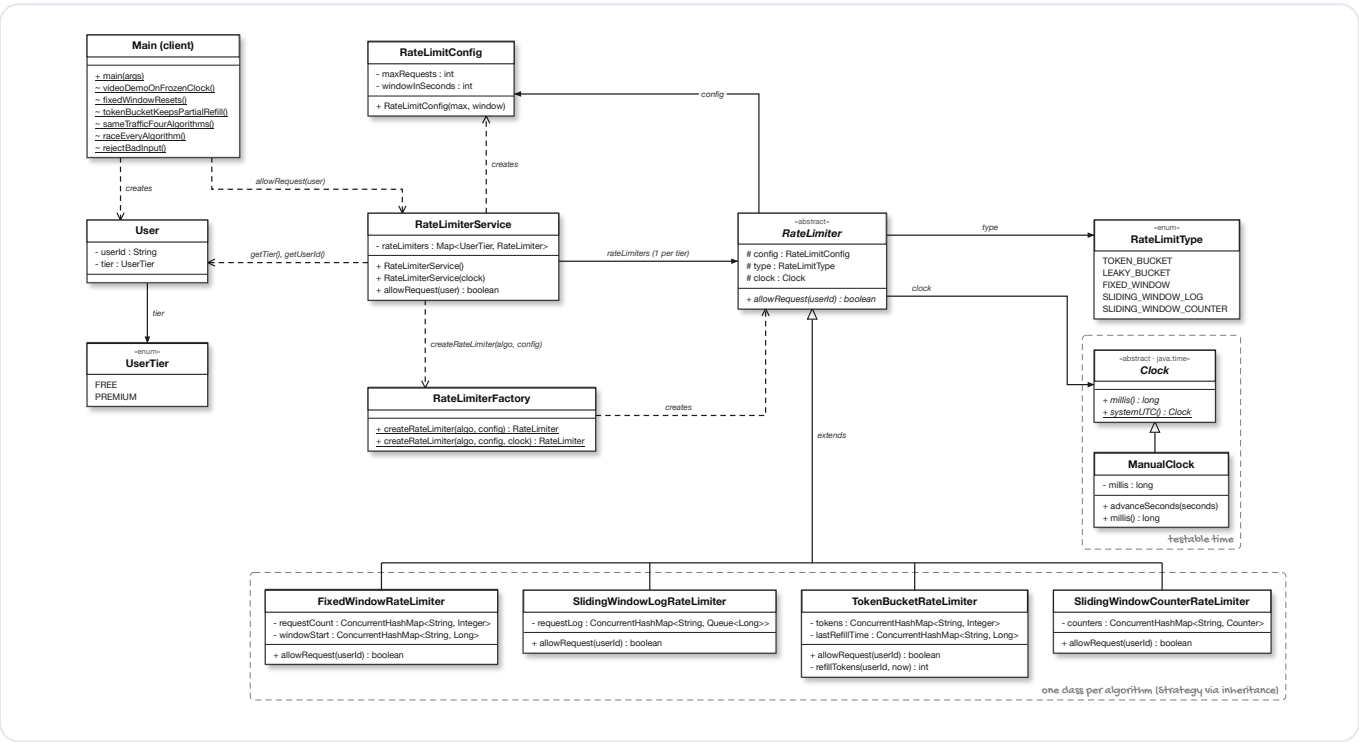
2 **RateLimiter & Fixed Window**
abstract class · window index

3 **Sliding Log & Token Bucket**
Strategy · polymorphism

4 **Factory & RateLimiterService**
Simple Factory · per-tier routing

5 **Thread Safety**
compute() · per-user locks

6 **Extensions**
Clock · 4th algorithm · 2 bug fixes



Source. "Design Rate Limiter (LLD) - Token Bucket, Fixed & Sliding Window with Thread Safety" by Shubh Patel (YouTube, 32 min), and its reference code at github.com/shubhkpatel/Low-Level-Design (`rate_limiter` package).

How to use these notes

We never draw the finished diagram first. We start with the smallest useful piece of the domain and add one idea per phase, always ending with **code that compiles and runs**. Each phase has its own folder with the complete code for that point in the design, so you can run it, diff it against the phase before, or retype it from memory.

In every phase you get

- ✓ **The goal:** which requirement this phase satisfies.
- ✓ **A UML diagram** of everything built so far. Classes added in the phase are tagged NEW and classes that gained or changed members are tagged UPDATED, with those members shaded.
- ✓ **Concepts and patterns** with code from that phase's folder. In code cards, lines that changed since the previous phase have a purple bar.
- ✓ **Real output** from running the phase.
- ✓ **A checkpoint**: questions to answer before you move on.

Contents

Understand the system: LeetCode's limiter, requirements, the four algorithms	3
The destination: the video's UML, how to read the diagrams, the pattern toolkit	7
Phase 1 · Users, Tiers & Limits	9
Phase 2 · RateLimiter & Fixed Window	12
Phase 3 · Sliding Log & Token Bucket	16
Phase 4 · Factory & RateLimiterService	20
Phase 5 · Thread Safety	24
Phase 6 · Extensions	30
The complete code flow (sequence diagrams)	36
The complete design	38
Notes on the reference code · where to take it next	39
Self-test interview questions	41

Folder map

```
RateLimiter-LLD-Study/
├── RateLimiter_LLD_Study_Notes.pdf  ← this file
├── README.md
├── Phase-1-Users-Tiers-and-Limits/
├── Phase-2-RateLimiter-and-Fixed-Window/
├── Phase-3-Sliding-Log-and-Token-Bucket/
├── Phase-4-Factory-and-RateLimiter-Service/
├── Phase-5-Thread-Safety/   ← the video's design
├── Phase-6-Extensions/      ← final design
├── diagrams/     SVG UML per phase + sequences
└── screenshots/  frames from the video
```

Each phase folder has `run.sh` and `src/ratelimiter/…`, split into the packages the reference uses: `model`, `enums`, `limiter`, `factory`, `service` (plus `clock` in Phase 6). You only need a JDK (17+): no Maven, no Lombok.

```
bash Phase-3-Sliding-Log-and-Token-Bucket/run.sh
diff -ru Phase-4-Factory-and-RateLimiter-Service/src \
         Phase-5-Thread-Safety/src
```

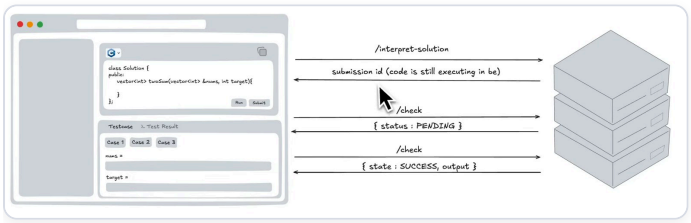
Phases 2 and 3 sleep for a few seconds to show time passing. Every other phase finishes in under a second.

Study loop that works

Read the phase → **sketch** its UML on paper before looking → **retype** the new classes into a copy of the previous phase (`cp -R Phase-2-… my-phase-3`) → **run** and `diff` against the real folder → answer the **checkpoint**. Retyping teaches more than rereading.

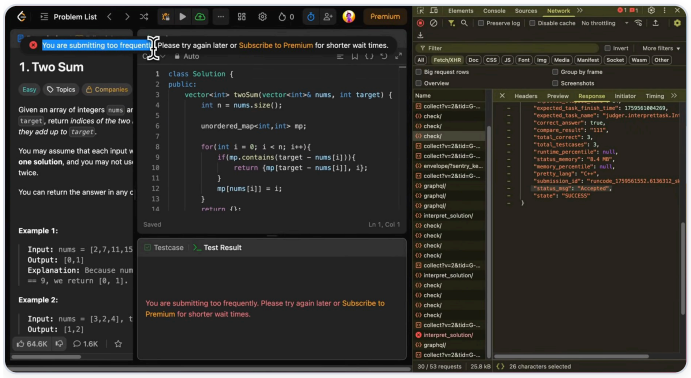
Step 0: Understand the system

The video opens on LeetCode. Pressing **Run** on a solution doesn't make one API call, it makes several:



Video @ 1:30 · one click on Run: a submit call, then repeated polling.

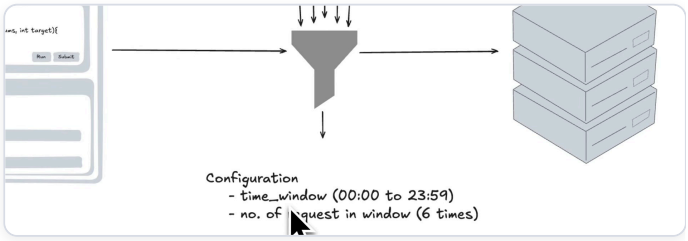
1. `/interpret_solution` sends the test cases, language, question id and code. Running the code takes 2–3 seconds, so the back end replies at once with a **submission id**.
2. The front end then **polls** `/check` until the state changes from PENDING to SUCCESS. In the demo it called `/check` four times.
3. So one click is already 5 requests. A user who spams Run, or a script that does it, multiplies the load on the servers until they fall over.



Video @ 4:05 · the third quick click is rejected: "You are submitting too frequently".

What a rate limiter is

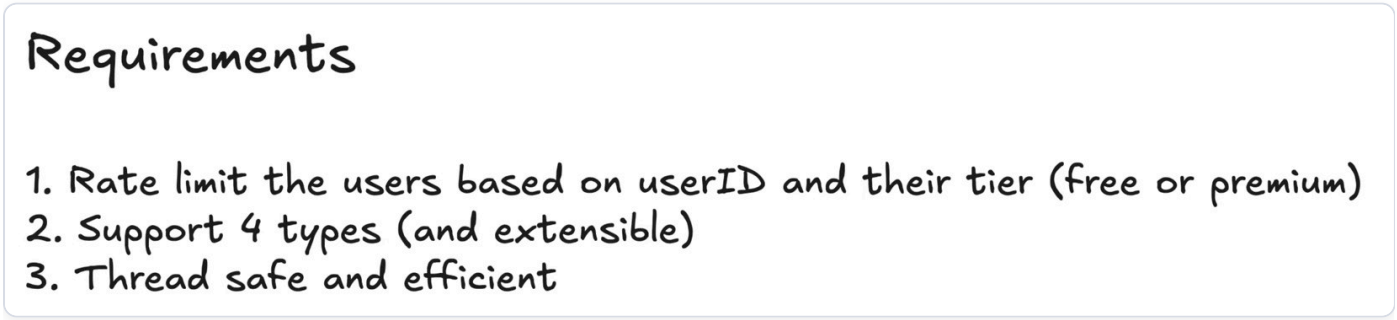
A gate in front of the servers that lets each client make at most **N requests per time window**. Requests over the limit are rejected before they reach the server, normally with **HTTP 429 Too Many Requests**. LeetCode does exactly this, and nudges you towards Premium, which gets shorter waits.



Video @ 4:40 · every limit is two numbers.

> **Interview tip**
>
> Clarify scope out loud: per-user limits, tiers, which algorithms, thread safety. Then say what you are *not* building in an LLD round: a distributed limiter shared by many servers (Redis), per-endpoint rules, an admin UI.

Requirements



Video @ 6:10 · the requirements board.

Three requirements, each with a hint about the design:

- "based on userID **and their tier**" means the limit belongs to the tier, and the state belongs to the user.
- "Support 4 types **(and extensible)**" means a fifth algorithm must not touch existing code: a Strategy family.
- "Thread safe **and efficient**" means no global lock. The video's example: if the API takes 200 ms and the limiter 2 s, every click takes 2.2 s.

From requirement to design decision

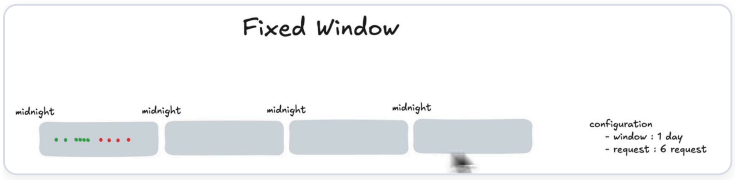
#	Requirement (from the video)	Design decision	Phase
R1	Rate limit users by user id and tier. Free is tight (the board's example: 2 requests in 10 s); premium is loose (10 in 10 s)	`User(userId, tier)`, `RateLimitConfig(maxRequests, windowInSeconds)`; `RateLimiterService` keeps one limiter per tier, and each limiter keeps one entry per `userId`	1 4
R2	Support 4 algorithms: token bucket, fixed window, sliding window log, sliding window counter. Adding a 5th must not modify existing code	Abstract `RateLimiter` with one subclass per algorithm, built by `RateLimiterFactory` from a `RateLimitType`	2 3 4 6
R3	Thread safe: no inaccuracy under concurrent requests	`ConcurrentHashMap.compute(userId, …)`: an atomic read-modify-write that locks only that user's entry	5
R4	Efficient: the limiter must add almost nothing to response time	O(1) work per request for fixed window, token bucket and sliding counter; no lock shared between users	2 5

Why is the limit on the tier and the state on the user?

Every free user gets the same rule, so there is one `RateLimitConfig` and one limiter object for the FREE tier. But each user has their own budget, so inside that limiter every map is keyed by `userId`. Adding a PREMIUM_PLUS tier (the video suggests it at 7:21) is one more map entry, not new classes.

The four algorithms, before any code

The video explains three algorithms with animations, using LeetCode's example limit of **6 requests per 24-hour day**. The fourth, sliding window counter, is on the requirements board but neither the video nor its code builds it. Phase 6 does.

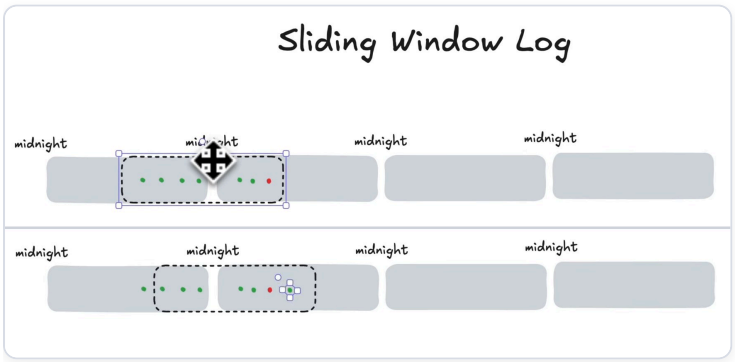


Video @ 11:50 · days are fixed windows: 6 green (allowed), the rest red (blocked).

1 · Fixed window

Cut time into fixed windows (midnight to midnight) and count requests in the current one. Send 100 requests on Monday and only the first 6 get through; the counter starts again at 0 on Tuesday.

Weakness: the windows ignore *when* inside them you send. 6 requests at 23:59 and 6 more at 00:00 are 12 requests in two minutes, double the limit (13:20).

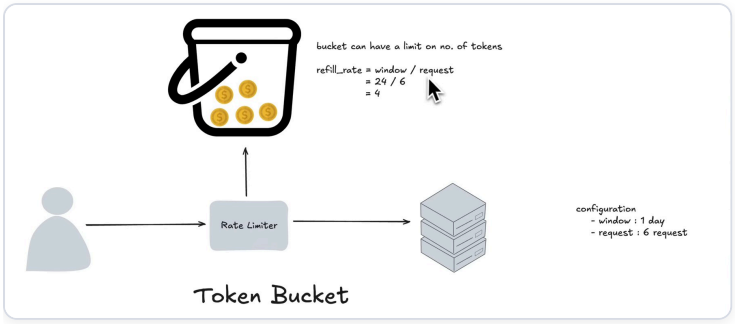


Video @ 14:25 and 14:33 · the dashed 24-hour window slides; once the oldest green dot leaves on the left, one more request fits.

2 · Sliding window log

Keep a log (a queue) of the timestamps of allowed requests. On each request, drop timestamps older than one window, then allow it only if fewer than 6 remain. No boundary burst is possible: any 24-hour span holds at most 6.

Weakness: memory. It stores every allowed request of the last window, for every user (15:43).



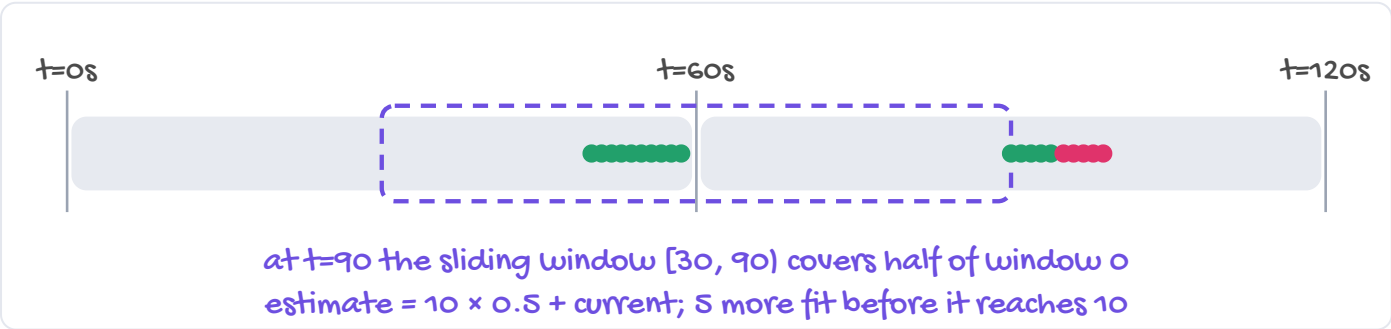
Video @ 17:03 · capacity = the max requests (6); refill rate = window / requests = 24 h / 6 = one token every 4 h.

3 · Token bucket

Each user has a bucket of tokens. A request takes one token; an empty bucket means blocked. Tokens drip back in at a fixed rate, and the bucket never holds more than its capacity.

It allows a short **burst** up to the capacity, then a steady average rate. The video's code uses it for free users.

4 · Sliding window counter (beyond the video)



Keep only two counters per user: this fixed window's count and the previous one's. Estimate the sliding window by assuming the previous window's requests were spread evenly:

```
estimate = previous × (share of the previous window still inside the sliding window) + current
```

O(1) memory like fixed window, and nearly as smooth as the log. The estimate is approximate.

Side by side (limit N per window W)

Algorithm	State per user	Memory	Bursts	Accuracy
Fixed window	count + window index	O(1)	up to 2N across a window boundary	exact per window, not per span
Sliding window log	queue of allowed timestamps	O(N)	none	exact for any span of length W
Token bucket	tokens + last refill time	O(1)	up to N at once, then N/W per second	controls the average rate, by design
Sliding window counter	2 counts + window index	O(1)	about none	approximate (assumes an even spread)

Say this in the interview

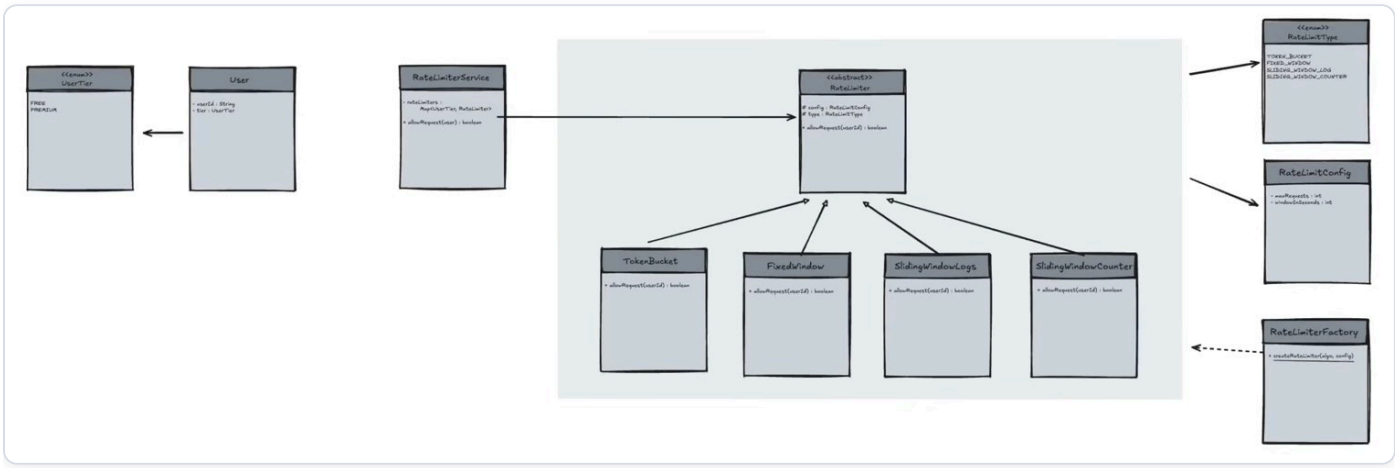
"Fixed window is the simplest but lets through double the limit at a boundary. The sliding log is exact but stores every request. Token bucket allows controlled bursts with O(1) state. The sliding window counter approximates the log with two counters."

Where each one fits

Token bucket when short bursts are fine but the average must hold (API quotas). Sliding log when the limit is small and must be exact (login attempts). Fixed window when simplicity matters most. Sliding counter for large limits that should still feel smooth.

The destination: the UML from the video

This is the whole class diagram the video draws (7:05). Don't try to memorise it: Phases 1–5 rebuild it piece by piece and Phase 6 extends it. The close-ups on each phase's pages come from the same whiteboard.



Video @ 7:05 · User and UserTier on the left, RateLimiterService in the middle, the abstract RateLimiter and its four algorithms in the shaded area, and RateLimitType, RateLimitConfig and RateLimiterFactory on the right.

How to read the diagrams in these notes

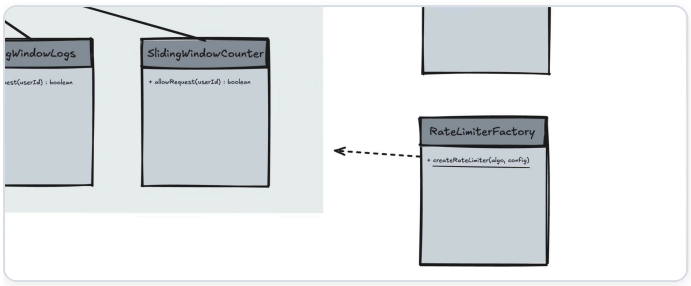
Inheritance : extends a class

Realization : implements an interface (not used here)

Association : keeps a reference in a field

Dependency : uses or creates, keeps no field

Members: `+` public, `-` private, `#` protected, `~` package-private, underlined = static, *italic* = abstract. «enum» and «abstract» are stereotypes. Getters are left out of the boxes. The algorithms' boxes show the map type actually created (HashMap or ConcurrentHashMap), because that is what changes in Phase 5.



Video @ 9:45 · RateLimiterFactory; its dashed arrow points into the algorithm family: it creates them but keeps none.

The flow the video walks through (10:07)

1. The client calls the API; the request first reaches `RateLimiterService.allowRequest(user)`.
2. The service checks the user's tier and picks that tier's limiter: tight for FREE, loose for PREMIUM.
3. The limiter is one of the concrete algorithms (never the abstract class itself), and decides by user id.
4. Allowed requests go on to the server; the rest get HTTP 429.

The factory is only used while wiring things up, which is why it has no arrow from the client.

Your toolkit: the patterns and principles in this design

Strategy, via an abstract class 2 3

Problem: one action (should this request pass?) has several algorithms, and more will come. **Shape:** abstract `RateLimiter` with one abstract method `allowRequest(userId)`; one subclass per algorithm. **Win:** callers hold a `RateLimiter` and never ask which one it is.

Simple Factory 4

Problem: the algorithm is chosen by an enum value in configuration; someone must turn it into an object. **Shape:** `RateLimiterFactory.createRateLimiter(type, config)` with one `switch`. **Win:** the only `switch` on `RateLimitType` is in one place (9:33).

Per-tier routing 4

Problem: free and premium users need different limits and maybe different algorithms. **Shape:** `Map<UserTier, RateLimiter>` inside `RateLimiterService`; `allowRequest(user)` looks up the tier and delegates. **Win:** a new tier is one map entry.

Per-key locking 5

Problem: two requests from the same user must not both read "1 token left"; requests from different users must not wait for each other. **Shape:** `ConcurrentHashMap.compute(userId, fn)` runs `fn` atomically for that key only. **Win:** correct and fast.

Inject the clock 6

Problem: code that calls `System.currentTimeMillis()` can only be tested by waiting. **Shape:** limiters read a `java.time.Clock`; tests pass a `ManualClock` they move by hand. **Win:** exact, instant tests of refills and window boundaries.

Immutable configuration 1

Problem: limits are shared by every request on every thread. **Shape:** `RateLimitConfig` and `User` have only `final` fields. **Win:** safe to share across threads with no locking at all.

SOLID principles you'll see again and again

Principle	Where it shows up
Single Responsibility	Each limiter runs one algorithm; `RateLimiterFactory` only builds; `RateLimiterService` only routes by tier.
Open/Closed	New algorithm = new `RateLimiter` subclass + one factory case. Phase 6 adds SLIDING_WINDOW_COUNTER this way.
Liskov Substitution	Phase 3's `Main` loops over three limiters through the `RateLimiter` type; all honour "true = let it through".
Interface Segregation	`RateLimiter` exposes exactly one public method.
Dependency Inversion	The service depends on `RateLimiter`, not on `TokenBucketRateLimiter`; in Phase 6 limiters depend on `Clock`, not on the system time.

PHASE 1

Users, Tiers & Limits

Entities · Enums · Immutable value objects

Model the nouns first: who is asking (a user with a tier) and what a limit is (N requests per W seconds). No limiting yet. The client keeps a map from tier to limit by hand.

Phase-1-Users-Tiers-and-Limits/

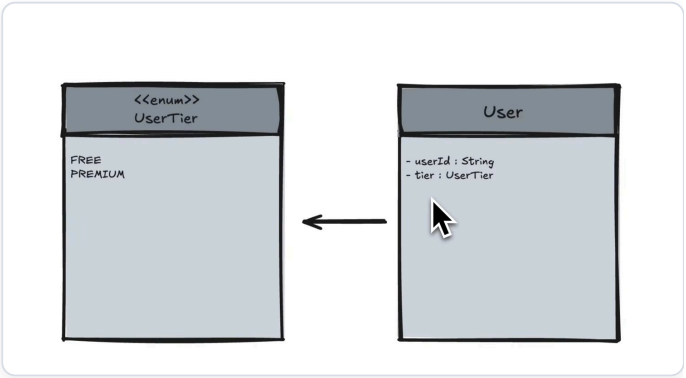
`User` `UserTier` `RateLimitConfig` `Main`

Goal

Requirement **R1** at the data level: every request comes from a `User` with an id and a `UserTier`, and every tier has a `RateLimitConfig`.

Concepts

- **Enum for the tier.** `UserTier` is a closed set (FREE, PREMIUM). It is data, not behaviour: the behaviour per tier arrives in Phase 4.
- **Value object.** `RateLimitConfig` is just two numbers, both `final`. Two configs with the same numbers mean the same limit.
- **Association.** `User` holds a `tier` field, so the diagram draws User → UserTier.
- **Derived numbers.** 10 per 60 s is one request every 6 s on average. The token bucket in Phase 3 refills at exactly that rate.



Video @ 7:16 · User (userId, tier) and the UserTier enum.

The new classes

enums/UserTier.java — NEW

```java
public enum UserTier {
    FREE,
    PREMIUM
}
```

model/RateLimitConfig.java — NEW

```java
public class RateLimitConfig {
    private final int maxRequests;
    private final int windowInSeconds;

    public RateLimitConfig(int maxRequests, int
     windowInSeconds) {
        this.maxRequests = maxRequests;
        this.windowInSeconds = windowInSeconds;
    }

    public int getMaxRequests() {
        return maxRequests;
    }

    public int getWindowInSeconds() {
        return windowInSeconds;
    }
}
```

model/User.java — NEW

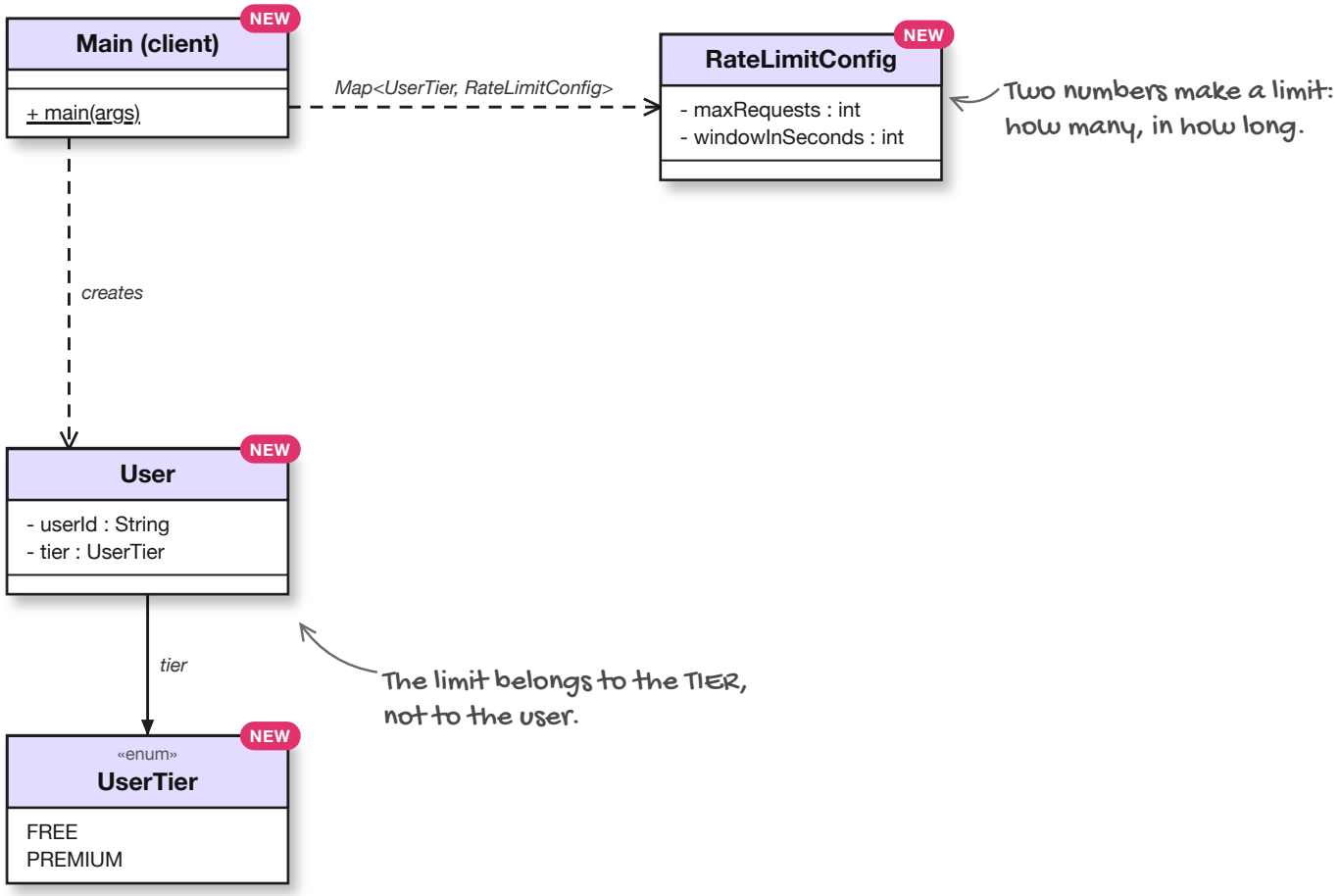
```java
public class User {
    private final String userId;
    private final UserTier tier;

    public User(String userId, UserTier tier) {
        this.userId = userId;
        this.tier = tier;
    }

    public String getUserId() {
        return userId;
    }

    public UserTier getTier() {
        return tier;
    }
}
```

Phase 1 UML: the domain model



New: `User`, `UserTier`, `RateLimitConfig`. `Main` holds a `Map<UserTier, RateLimitConfig>`; in Phase 4 that map moves into `RateLimiterService`.

Main.java NEW

```java
public class Main {
    public static void main(String[] args) {
        User freeUser = new User("user1", UserTier.FREE);       // 10 req in 60 sec
        User premiumUser = new User("user2", UserTier.PREMIUM); // 100 req in 60 sec

        // Every rate limit is two numbers: how many requests, in how long a window.
        // The limit belongs to the tier, not to the user.
        Map<UserTier, RateLimitConfig> limits = new LinkedHashMap<>();
        limits.put(UserTier.FREE, new RateLimitConfig(10, 60));
        limits.put(UserTier.PREMIUM, new RateLimitConfig(100, 60));

        System.out.println("=== Limit per tier ===");
        for (Map.Entry<UserTier, RateLimitConfig> entry : limits.entrySet()) {
            System.out.println(entry.getKey() + ": " + describe(entry.getValue()));
        }

        System.out.println("\n=== Which limit applies to each user? ===");
        for (User user : List.of(freeUser, premiumUser)) {
            RateLimitConfig limit = limits.get(user.getTier());
            System.out.println(user.getUserId() + " (" + user.getTier() + ") → " + describe(limit));
        }

        // The LeetCode example from the video: 6 runs per 24-hour day.
        RateLimitConfig leetCode = new RateLimitConfig(6, 24 * 60 * 60);
        System.out.println("\n=== The video's LeetCode example ===");
        System.out.println("6 per day: " + describe(leetCode) + " = one every "
                + leetCode.getWindowInSeconds() / leetCode.getMaxRequests() / 3600 + " hours");
    }

    // "10 requests / 60 s (one every 6.0 s on average)"
    private static String describe(RateLimitConfig config) {
        double gap = (double) config.getWindowInSeconds() / config.getMaxRequests();
        return config.getMaxRequests() + " requests / " + config.getWindowInSeconds() + " s"
                + String.format(" (one every %.1f s on average)", gap);
    }
}
```

```
▶ bash Phase-1-Users-Tiers-and-Limits/run.sh

=== Limit per tier ===
FREE: 10 requests / 60 s (one every 6.0 s on average)
PREMIUM: 100 requests / 60 s (one every 0.6 s on average)

=== Which limit applies to each user? ===
user1 (FREE) → 10 requests / 60 s (one every 6.0 s on average)
user2 (PREMIUM) → 100 requests / 60 s (one every 0.6 s on average)

=== The video's LeetCode example ===
6 per day: 6 requests / 86400 s (one every 14400.0 s on average) = one every 4 hours
```

Why `final` everywhere?

From Phase 5 on, many threads read the same `User` and `RateLimitConfig` at once. Objects whose fields are all `final` can be shared between threads with no locks. Keep mutable state in as few places as possible: here, only inside the limiters.

In the video @ 4:33

The video introduces the two numbers with LeetCode: "only request six times in an entire day". The last line of the output is that limit: one request every 4 hours on average, the token bucket refill rate the video computes at 16:50.

Checkpoint

1) Why is `tier` a field of `User` rather than a `Map<String, UserTier>` somewhere else?
2) Where will per-user state (counts, tokens) live, given that configs are per tier? *(Inside each limiter, in maps keyed by userId.)*
3) What would PREMIUM_PLUS cost? *(One enum value and one config.)*

PHASE 2

RateLimiter & Fixed Window

Abstract class · Window index · Per-user state

Introduce the contract every algorithm will follow, then implement the simplest algorithm: count requests per user in fixed windows of time. Single-threaded for now; Phase 5 makes it thread-safe.

Phase-2-RateLimiter-and-Fixed-Window/

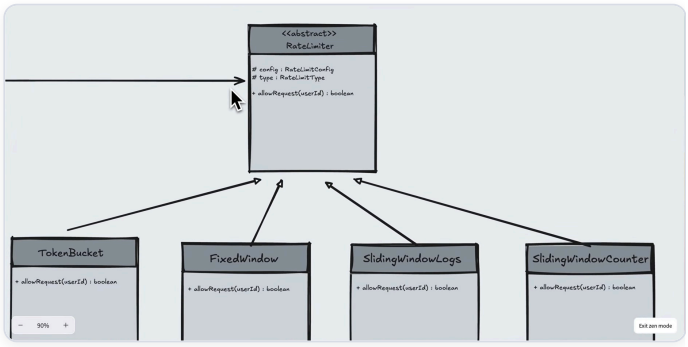
`RateLimiter «abstract»` `FixedWindowRateLimiter` `RateLimitType`

Goal

Requirement **R2**, first step: a common type for all algorithms, and the first algorithm behind it.

The contract

`RateLimiter` is **abstract** with one abstract method, `boolean allowRequest(String userId)`. It also holds what every algorithm needs: its `config` and its `type`. Both are `protected final`, so subclasses can read them but never replace them.



Video @ 9:02 · «abstract» RateLimiter with # config and # type; each algorithm overrides allowRequest(userId).

The fixed-window algorithm

1. **Which window are we in?** `now / 1000 / windowInSeconds`. With a 60 s window, every timestamp from 12:00:00 to 12:00:59 gives the same number.
2. **Same window as this user's last request?** Then increment the user's count if it is still below the limit.
3. **A new window?** Then reset: remember the new window, set the count to 1, allow.

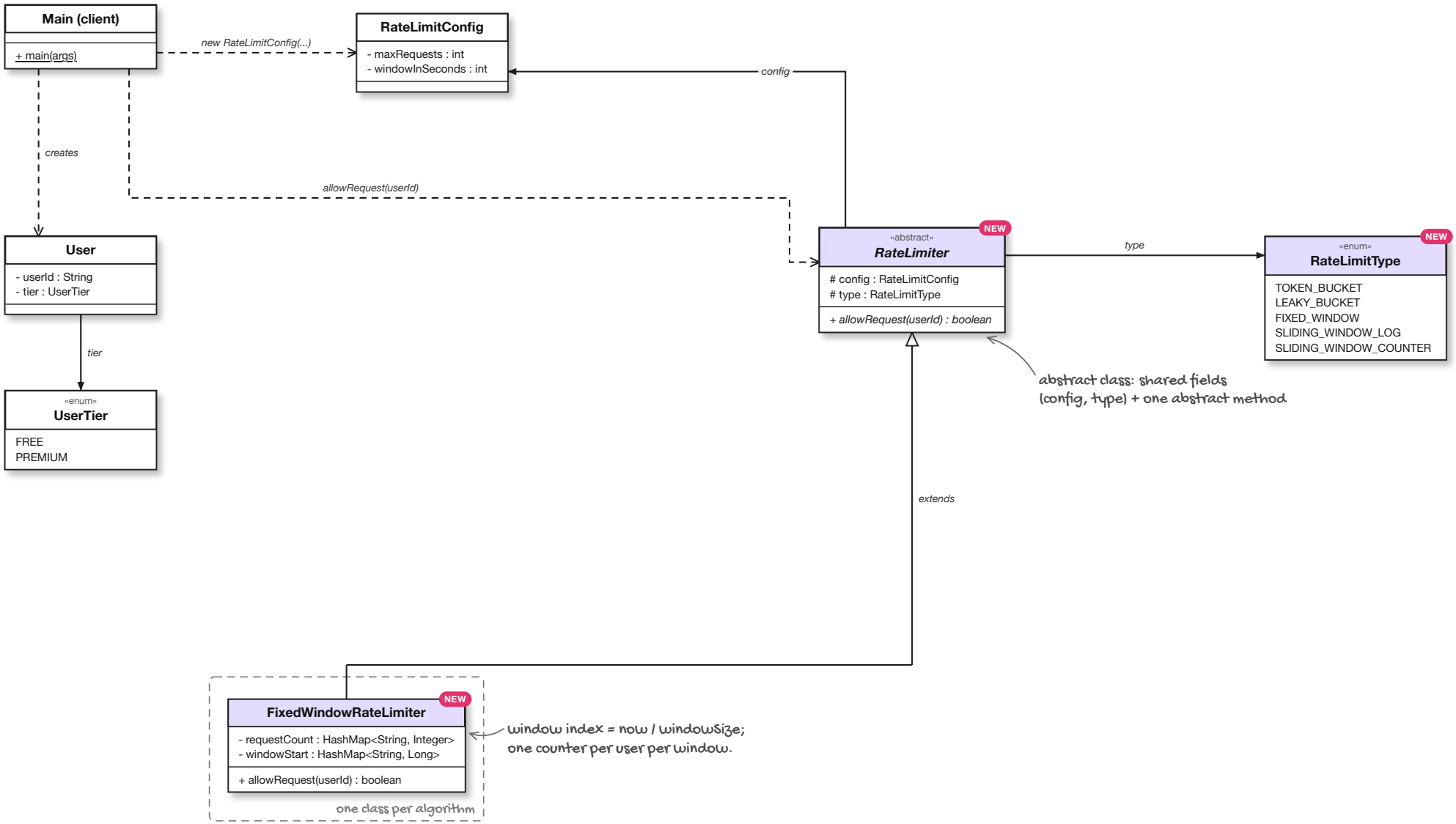
Abstract class or interface?

The video uses an abstract class because every algorithm shares two fields and a constructor. An interface plus a constructor parameter in each class would work too. Say which you chose and why. The deciding question is "is there shared *state*?" Here there is: `config` and `type`.

Why keyed by `userId` and not by `User`?

The limiter only needs a stable key. A `String` has proper `equals` / `hashCode`; `User` does not override them, so two `User` objects for the same person would be two different keys.

Phase 2 UML: the contract and the first algorithm



New: «abstract» `RateLimiter`, `FixedWindowRateLimiter` and the `RateLimitType` enum (all five values from the reference, including LEAKY_BUCKET, which nobody implements). The hollow triangle means *extends*.

enums/RateLimitType.java NEW

```java
public enum RateLimitType {
    TOKEN_BUCKET,
    LEAKY_BUCKET,
    FIXED_WINDOW,
    SLIDING_WINDOW_LOG,
    SLIDING_WINDOW_COUNTER
}
```

limiter/RateLimiter.java NEW

```java
public abstract class RateLimiter {
    protected final RateLimitConfig config;
    protected final RateLimitType type;

    public RateLimiter(RateLimitConfig config,
     RateLimitType type) {
        this.config = config;
        this.type = type;
    }

    public abstract boolean allowRequest(String userId);
}
```

limiter/FixedWindowRateLimiter.java NEW

```java
// Single-threaded version. Phase 5 makes it thread-safe.
public class FixedWindowRateLimiter extends RateLimiter {
    private final Map<String, Integer> requestCount = new HashMap<>();
    private final Map<String, Long> windowStart = new HashMap<>();

    public FixedWindowRateLimiter(RateLimitConfig config) {
        super(config, RateLimitType.FIXED_WINDOW);
    }

    @Override
    public boolean allowRequest(String userId) {
        long currentReqWindow = System.currentTimeMillis() / 1000 / config.getWindowInSeconds();

        long lastReqWindow = windowStart.getOrDefault(userId, currentReqWindow);

        if (lastReqWindow ≠ currentReqWindow) {
            // window expired → reset counter and window of last req
            windowStart.put(userId, currentReqWindow);
            requestCount.put(userId, 1); // first request in new window
            return true;
        }

        int count = requestCount.getOrDefault(userId, 0);

        if (count < config.getMaxRequests()) {
            requestCount.put(userId, count + 1); // increment count
            return true;
        }

        return false; // count remains same
    }
}
```

This is the reference algorithm, with one bug the video misses. Look at the line that reads `windowStart` and ask: when is `windowStart.put` ever reached for a brand-new user? The answer is in Phase 6.

Main.java (main method) CHANGED

```java
    public static void main(String[] args) throws InterruptedException {
        User freeUser = new User("user1", UserTier.FREE);
        User otherFreeUser = new User("user3", UserTier.FREE);

        RateLimiter limiter = new FixedWindowRateLimiter(new RateLimitConfig(10, 60)); // 10 req/min

        System.out.println("=== FixedWindowRateLimiter, 10 requests per 60 s ===");
        System.out.println("user1 sends 15: " + fire(limiter, freeUser, 15));
        System.out.println("user3 sends 3:  " + fire(limiter, otherFreeUser, 3) + "   (own counter)");

        // To see a window reset without waiting a minute, use 2 requests per 1-second window.
        RateLimiter perSecond = new FixedWindowRateLimiter(new RateLimitConfig(2, 1));

        System.out.println("\n=== FixedWindowRateLimiter, 2 requests per 1 s ===");
        System.out.println("user1 sends 3:  " + fire(perSecond, freeUser, 3));
        Thread.sleep(1100); // now we are in a later 1-second window
        boolean fourth = perSecond.allowRequest(freeUser.getUserId());
        System.out.println("1.1 s later, request 4: " + (fourth ? "ALLOWED" : "BLOCKED")
                + "   <- a new window has started. Expected ALLOWED: spot the bug!");
    }
```

```
▶ bash Phase-2-RateLimiter-and-Fixed-Window/run.sh

═══ FixedWindowRateLimiter, 10 requests per 60 s ═══
user1 sends 15: 1-10 ALLOWED, 11-15 BLOCKED
user3 sends 3:  1-3 ALLOWED   (own counter)

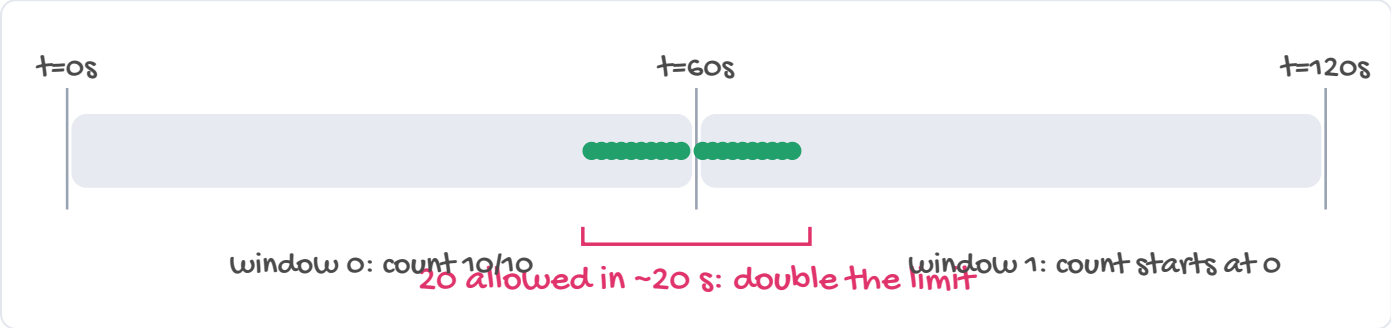
═══ FixedWindowRateLimiter, 2 requests per 1 s ═══
user1 sends 3:  1-2 ALLOWED, 3 BLOCKED
1.1 s later, request 4: BLOCKED   ← a new window has started. Expected ALLOWED: spot the bug!
```

Trace: the video's example (6 requests per 24-hour day)

When	Window index (day number)	Count before	Result
Monday, requests 1–6	d	0 … 5	ALLOWED, count → 6
Monday, requests 7–100	d	6	BLOCKED, count stays 6
Tuesday, request 1	d + 1 (new window)	6	should reset to 1 and allow…

The last row is where the reference code goes wrong for a user whose *first* window is still recorded nowhere. Our output shows it: 1.1 s later the 2-per-second limiter still says BLOCKED. Phase 6 proves it with a controllable clock and fixes it with one line.

The boundary burst (13:20)



With 10 per 60 s: 10 requests just before t=60 and 10 just after are all allowed, because they fall in different windows.

Not thread-safe yet

Two threads can both read count 9, both write 10 and both return true: 11 requests get through. Phase 5 shows this happening and fixes it. Building it single-threaded first lets you see exactly what thread safety adds.

Checkpoint

1) For a 60 s window, which window index does `t = 1,700,000,039,999 ms` fall in? And `t + 1 ms`? *(28,333,333 and 28,333,334: a new window.)*
2) Why does user3 get its own 10 requests in the output?
3) Find the bug before Phase 6: trace `windowStart` for a user's first two requests.

PHASE 3

Sliding Log & Token Bucket

Strategy · Polymorphism · Queue · Refill rate

Add two more algorithms behind the same contract. The client runs one burst of traffic through all three and never asks which algorithm it holds.

Phase-3-Sliding-Log-and-Token-Bucket/

`SlidingWindowLogRateLimiter` `TokenBucketRateLimiter`

Sliding window log

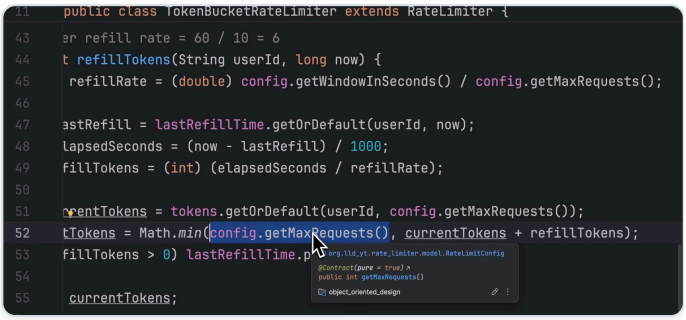
1. `now` in seconds.
2. From the front of the user's `Queue<Long>`, drop every timestamp with `now - t ≥ window`: those requests have slid out.
3. If fewer than `maxRequests` remain, record `now` and allow; otherwise block. Blocked requests are **not** recorded.

Token bucket

1. **Refill:** `refillRate = window / maxRequests` seconds per token (60 / 10 = 6 s for free users). Tokens earned = `elapsed seconds since last refill / refillRate`, capped at `maxRequests`.
2. If at least one token, take it and allow; otherwise block.
3. A user's first request finds no entry, so they start with a full bucket (`getOrDefault(userId, maxRequests)`).

The video's refill example (21:52)

Step	Value
last refill	t = 2 s
this request	t = 14 s
elapsed	14 − 2 = 12 s
refill rate (free)	60 / 10 = 6 s per token
tokens added	**12 / 6 = 2 (then capped at 10)**



Video @ 23:05 · refillTokens: elapsed seconds / refill rate, capped at maxRequests.

limiter/SlidingWindowLogRateLimiter.java — NEW

```java
// Single-threaded version. Phase 5 makes it thread-safe.
public class SlidingWindowLogRateLimiter extends RateLimiter {
    private final Map<String, Queue<Long>> requestLog = new HashMap<>();

    public SlidingWindowLogRateLimiter(RateLimitConfig config) {
        super(config, RateLimitType.SLIDING_WINDOW_LOG);
    }

    @Override
    public boolean allowRequest(String userId) {
        long now = System.currentTimeMillis() / 1000;

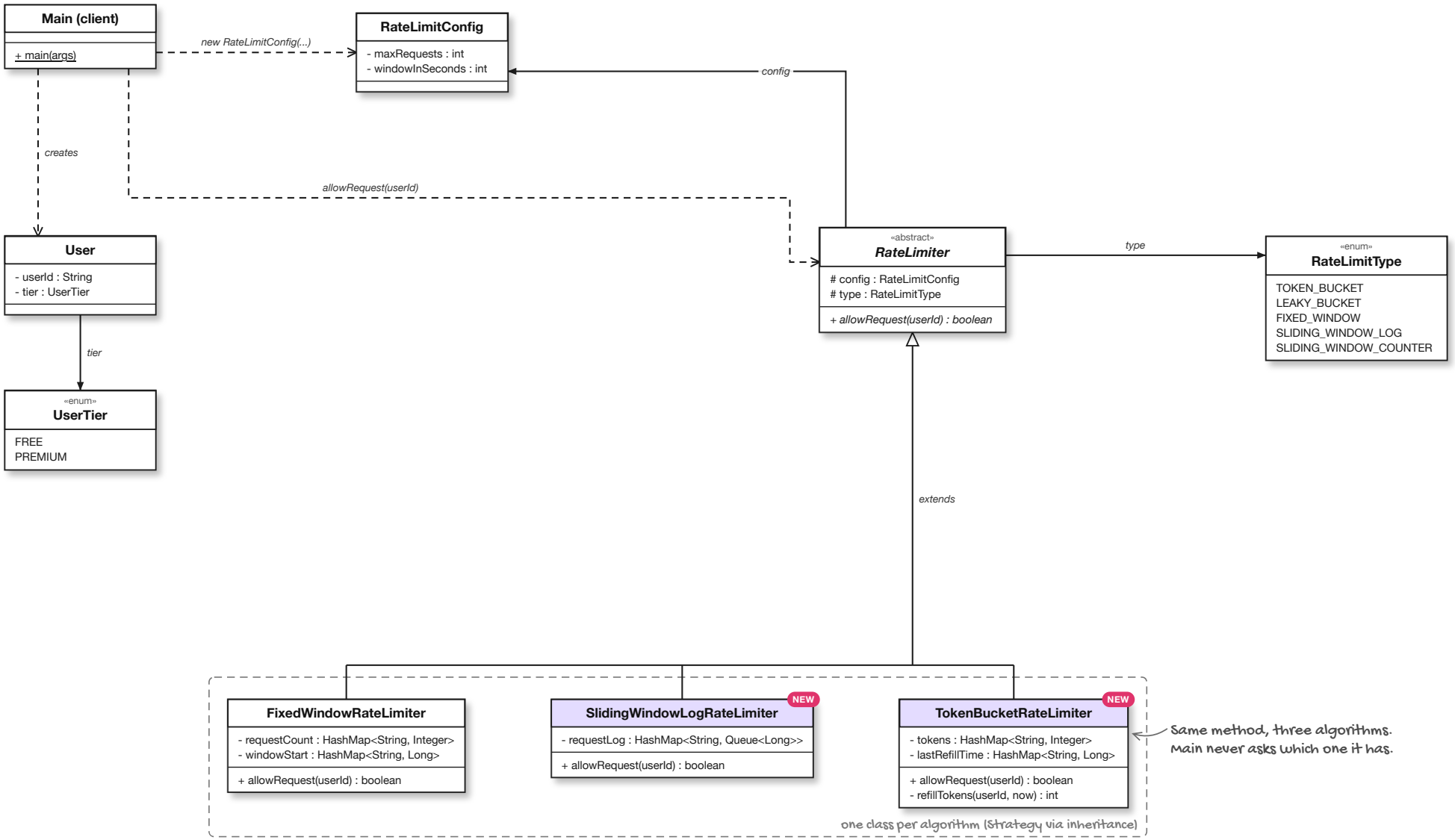
        Queue<Long> log = requestLog.computeIfAbsent(userId, id -> new ArrayDeque<>());

        // slide the window: drop timestamps that are a full window old
        while (!log.isEmpty() && (now - log.peek()) ≥ config.getWindowInSeconds()) {
            log.poll();
        }

        if (log.size() < config.getMaxRequests()) {
            log.add(now); // record this request
            return true;
        }

        return false;
    }
}
```

Phase 3 UML: three algorithms, one contract



New: `SlidingWindowLogRateLimiter` and `TokenBucketRateLimiter`. Nothing else changed: that is Open/Closed in action.

limiter/TokenBucketRateLimiter.java — NEW

```java
// Single-threaded version. Phase 5 makes it thread-safe.
public class TokenBucketRateLimiter extends RateLimiter {
    private final Map<String, Integer> tokens = new HashMap<>();
    private final Map<String, Long> lastRefillTime = new HashMap<>();

    public TokenBucketRateLimiter(RateLimitConfig config) {
        super(config, RateLimitType.TOKEN_BUCKET);
    }

    @Override
    public boolean allowRequest(String userId) {
        long now = System.currentTimeMillis();

        int currentTokens = refillTokens(userId, now);

        if (currentTokens > 0) {
            tokens.put(userId, currentTokens - 1); // consume 1 token
            return true;
        }

        tokens.put(userId, currentTokens); // remain at 0
        return false;
    }

    // free user refill rate = 60 / 10 = 6
    private int refillTokens(String userId, long now) {
        double refillRate = (double) config.getWindowInSeconds() / config.getMaxRequests();

        lastRefillTime.putIfAbsent(userId, now);
        long lastRefill = lastRefillTime.get(userId);

        long elapsedSeconds = (now - lastRefill) / 1000;
        int refillTokens = (int) (elapsedSeconds / refillRate);
        int currentTokens = tokens.getOrDefault(userId, config.getMaxRequests());
        currentTokens = Math.min(config.getMaxRequests(), currentTokens + refillTokens);

        if (refillTokens > 0) lastRefillTime.put(userId, now);
        return currentTokens;
    }
}
```

Main.java (main and round) — CHANGED

```java
    public static void main(String[] args) throws InterruptedException {
        User freeUser = new User("user1", UserTier.FREE);

        // A small limit, so the differences show up within a few seconds.
        RateLimitConfig config = new RateLimitConfig(3, 3); // 3 requests per 3 s
        List<RateLimiter> limiters = List.of(
                new FixedWindowRateLimiter(config),
                new SlidingWindowLogRateLimiter(config),
                new TokenBucketRateLimiter(config));

        System.out.println("=== Same config (3 requests / 3 s), three algorithms ===");
        round("t = 0.0 s, 5 requests each", limiters, freeUser, 5);
        Thread.sleep(1200);
        round("t = 1.2 s, 2 requests each", limiters, freeUser, 2);
        Thread.sleep(2400);
        round("t = 3.6 s, 3 requests each", limiters, freeUser, 3);
    }
```

```
▶ bash Phase-3-Sliding-Log-and-Token-Bucket/run.sh

═══ Same config (3 requests / 3 s), three algorithms ═══
t = 0.0 s, 5 requests each
  FixedWindowRateLimiter       1-3 ALLOWED, 4-5 BLOCKED
  SlidingWindowLogRateLimiter  1-3 ALLOWED, 4-5 BLOCKED
  TokenBucketRateLimiter       1-3 ALLOWED, 4-5 BLOCKED
t = 1.2 s, 2 requests each
  FixedWindowRateLimiter       1-2 BLOCKED
  SlidingWindowLogRateLimiter  1-2 BLOCKED
  TokenBucketRateLimiter       1 ALLOWED, 2 BLOCKED
t = 3.6 s, 3 requests each
  FixedWindowRateLimiter       1-3 BLOCKED
  SlidingWindowLogRateLimiter  1-3 ALLOWED
  TokenBucketRateLimiter       1-2 ALLOWED, 3 BLOCKED
```

Reading the output (limit: 3 requests per 3 s)

Time	Fixed window	Sliding window log	Token bucket (1 token / s)
0.0 s, 5 requests	3 allowed, count 3	3 timestamps logged	bucket 3 → 0
1.2 s, 2 requests	blocked (the bug: never resets)	blocked: the 3 timestamps are only 1–2 s old	1 s elapsed → 1 token → 1 allowed
3.6 s, 3 requests	blocked (same bug)	all 3 timestamps ≥ 3 s old → evicted → 3 allowed	2 s since the last refill → 2 tokens → 2 allowed

The sleeps were chosen so the results don't depend on where the run starts inside a second, so the output is the same every time. A correct fixed window would allow requests again once a new 3-second window started; with the bug it never does.

Why `List<RateLimiter>` works

`round()` only knows the abstract type. Each call to `allowRequest` runs whichever subclass the object really is: that is dynamic dispatch, and it is why the service in Phase 4 can hold one `RateLimiter` per tier without caring which algorithm it is.

Checkpoint

1) What does the sliding log store for a user who sent 100 requests and was allowed 10? *(10 timestamps.)*
2) With 100 requests per 60 s, how often does a token refill? *(Every 0.6 s.)*
3) Why does `elapsedSeconds` being a whole number matter for a 0.6 s refill rate?

PHASE 4

Factory & RateLimiterService

Simple Factory · Per-tier routing · The video's demo

Stop newing algorithms in the client. A factory turns a RateLimitType into a limiter; a service owns one limiter per tier and is the only thing the client calls.

Phase-4-Factory-and-RateLimiter-Service/

`RateLimiterFactory` `RateLimiterService`

Goal

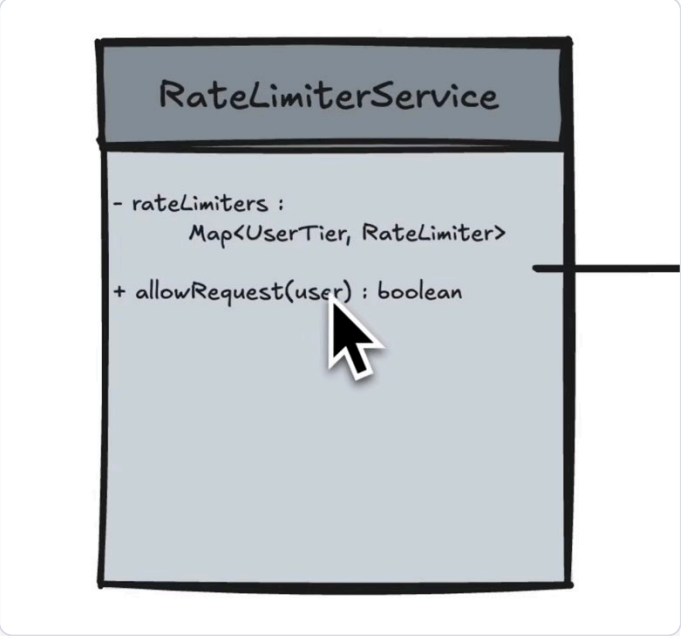
Requirement **R1** end to end: `allowRequest(user)` picks the limiter for the user's tier and asks it about the user's id. This is the design the video draws, with single-threaded algorithms for one more phase.

The service

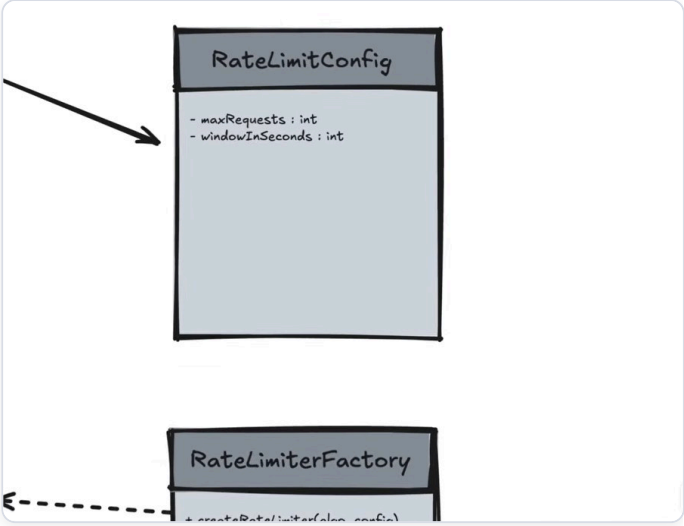
- The constructor wires the tiers: FREE → token bucket, 10 per 60 s; PREMIUM → fixed window, 100 per 60 s (19:04).
- `allowRequest(user)`: `rateLimiters.get(user.getTier())`, fail loudly if a tier has no limiter, then delegate with `user.getUserId()`.
- The map is a plain `HashMap` and that is fine: it is filled in the constructor, never changed again, and held in a `final` field, so every thread sees it fully built.

The factory

A `switch` expression on `RateLimitType`. LEAKY_BUCKET and SLIDING_WINDOW_COUNTER have no class, so they fall into `default` and throw.

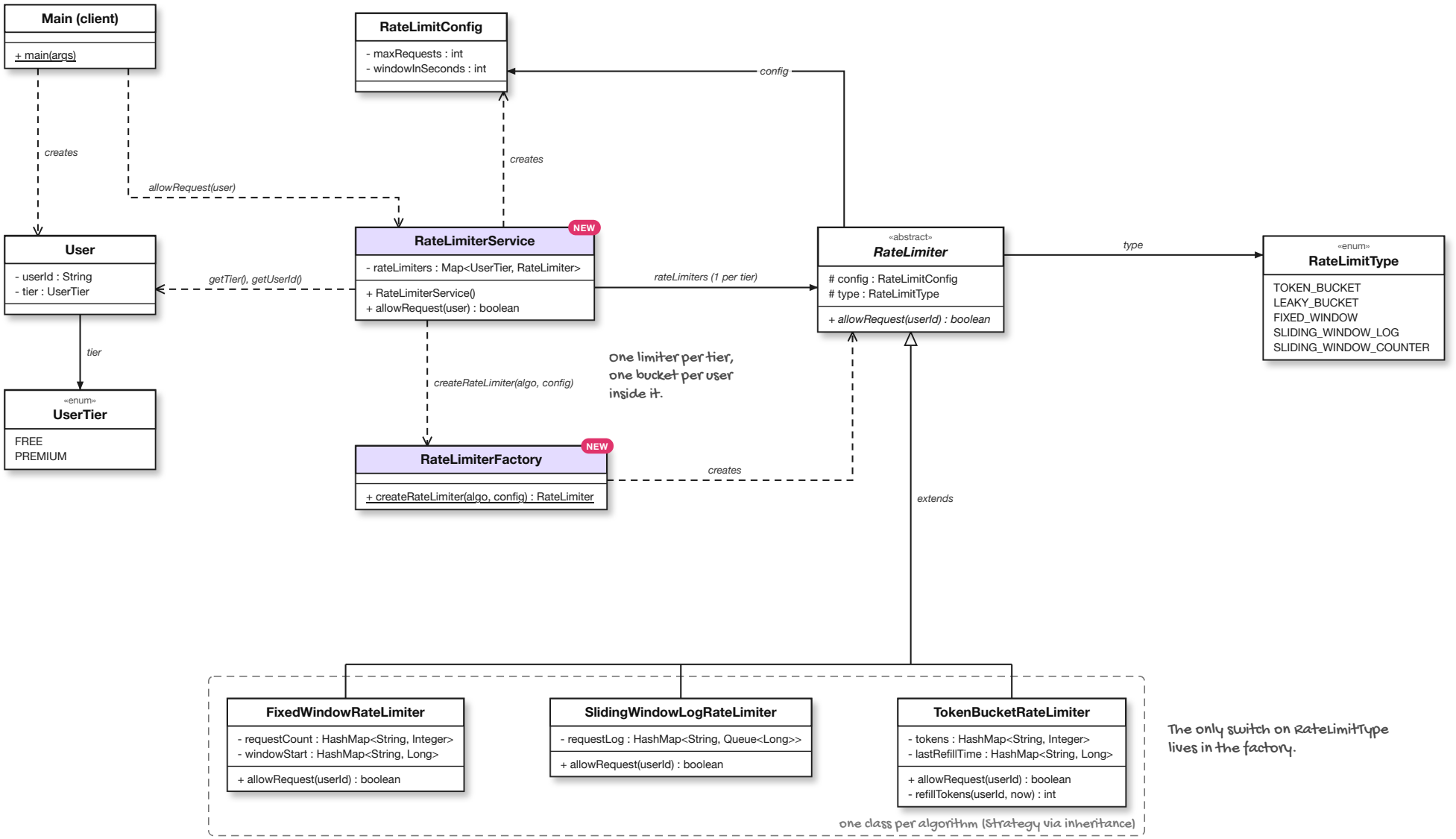


Video @ 7:50 · RateLimiterService: a Map<UserTier, RateLimiter> and allowRequest(user).



Video @ 8:56 · RateLimitConfig, and RateLimiterFactory with a static createRateLimiter(algo, config).

Phase 4 UML: the video's design



New: `RateLimiterService` and `RateLimiterFactory`. The client now talks only to the service; the service creates configs, asks the factory for limiters, and keeps one per tier.

factory/RateLimiterFactory.java — NEW

```
public class RateLimiterFactory {
    public static RateLimiter createRateLimiter(RateLimitType algo, RateLimitConfig config) {
        return switch (algo) {
            case TOKEN_BUCKET → new TokenBucketRateLimiter(config);
            case FIXED_WINDOW → new FixedWindowRateLimiter(config);
            case SLIDING_WINDOW_LOG → new SlidingWindowLogRateLimiter(config);
            default → throw new IllegalArgumentException("Unknown algorithm: " + algo);
        };
    }
}
```

service/RateLimiterService.java — NEW

```
public class RateLimiterService {
    private final Map<UserTier, RateLimiter> rateLimiters = new HashMap<>();

    public RateLimiterService() {
        // Configure per-tier limits + algorithms
        rateLimiters.put(
                UserTier.FREE,
                RateLimiterFactory.createRateLimiter(
                        RateLimitType.TOKEN_BUCKET,
                        new RateLimitConfig(10, 60) // 10 req/min
                )
        );

        rateLimiters.put(
                UserTier.PREMIUM,
                RateLimiterFactory.createRateLimiter(
                        RateLimitType.FIXED_WINDOW,
                        new RateLimitConfig(100, 60) // 100 req/min
                )
        );
    }

    public boolean allowRequest(User user) {
        RateLimiter limiter = rateLimiters.get(user.getTier());
        if (limiter == null) {
            throw new IllegalArgumentException("No limiter configured for tier: " + user.getTier());
        }
        return limiter.allowRequest(user.getUserId());
    }
}
```

Main.java (main method) — CHANGED

```
    public static void main(String[] args) {
        RateLimiterService rateLimiterService = new RateLimiterService();

        User freeUser = new User("user1", UserTier.FREE);       // 10 req in 60 sec
        User premiumUser = new User("user2", UserTier.PREMIUM); // 100 req in 60 sec

        System.out.println("=== Free User Requests ===");
        for (int i = 1; i <= 15; i++) {
            boolean allowed = rateLimiterService.allowRequest(freeUser);
            System.out.println("Request " + i + " for Free User: " + (allowed ? "ALLOWED" : "BLOCKED"));
        }

        System.out.println("\n=== Premium User Requests ===");
        System.out.println("120 requests: " + fire(rateLimiterService, premiumUser, 120));

        // One limiter per tier, but one bucket per user inside it.
        User anotherFreeUser = new User("user3", UserTier.FREE);
        System.out.println("\n=== Another free user shares the limiter, not the bucket ===");
        System.out.println("user3 sends 12: " + fire(rateLimiterService, anotherFreeUser, 12));

        System.out.println("\n=== The factory only builds what exists ===");
        try {
            RateLimiterFactory.createRateLimiter(RateLimitType.LEAKY_BUCKET, new RateLimitConfig(10, 60));
        } catch (IllegalArgumentException e) {
            System.out.println("LEAKY_BUCKET → " + e.getMessage());
        }
    }
```

```
▶ bash Phase-4-Factory-and-RateLimiter-Service/run.sh

═══ Free User Requests ═══
Request 1 for Free User: ALLOWED
Request 2 for Free User: ALLOWED
Request 3 for Free User: ALLOWED
Request 4 for Free User: ALLOWED
Request 5 for Free User: ALLOWED
Request 6 for Free User: ALLOWED
Request 7 for Free User: ALLOWED
Request 8 for Free User: ALLOWED
Request 9 for Free User: ALLOWED
Request 10 for Free User: ALLOWED
Request 11 for Free User: BLOCKED
Request 12 for Free User: BLOCKED
Request 13 for Free User: BLOCKED
Request 14 for Free User: BLOCKED
Request 15 for Free User: BLOCKED

═══ Premium User Requests ═══
120 requests: 1-100 ALLOWED, 101-120 BLOCKED

═══ Another free user shares the limiter, not the bucket
═══
user3 sends 12: 1-10 ALLOWED, 11-12 BLOCKED

═══ The factory only builds what exists ═══
LEAKY_BUCKET → Unknown algorithm: LEAKY_BUCKET
```

In the video @ 29:52

The video runs exactly this: the free user gets requests 1–10 and is blocked for 11–15; the premium user gets 1–100 and is blocked for 101–120. The video sleeps 100 ms between requests; these notes don't, so the run takes milliseconds.

Why is user3 allowed 10?

user1 and user3 share the FREE tier's `TokenBucketRateLimiter` object, but its maps are keyed by user id, so each has a separate bucket. One limiter per tier, one bucket per user.

Checkpoint

1) Swap free users to the sliding window log. What do you change? *(One argument in the service constructor.)*
2) Why does the factory method not need to be an instance method?
3) What happens to a request from a user whose tier has no limiter?

PHASE 5

Thread Safety

ConcurrentHashMap.compute · Per-key locking · AtomicBoolean · CyclicBarrier

Many requests arrive at once. Make each algorithm's read-modify-write atomic per user, without one big lock, and prove it with a race test. This phase is the reference code.

Phase-5-Thread-Safety/

`3 limiters UPDATED` `checkConcurrency` `raceManyTimes`

The race

Every algorithm does **read → decide → write**: read the tokens, decide, write tokens − 1. If two threads for the same user interleave, both read 1 token, both decide "allowed", both write 0. Two requests got through on one token.

Three ways to fix it

`synchronized allowRequest`	correct, but every user waits for every other user
a lock per user	correct and parallel, but you manage a map of locks
`map.compute(userId, fn)`	the map does the per-key locking for you. The video's choice (21:18).

What `compute` guarantees

`ConcurrentHashMap.compute(key, fn)` runs `fn(key, oldValue)` and stores its result **atomically**: no other `compute` on the same key can run in between. Calls for other keys proceed in parallel. (It locks the key's hash bin, so two keys that share a bin can briefly wait for each other.)

In the video @ 20:44

The video asks whether requests from user one and user two need to be synchronized with each other. No: each has its own map entry. Only simultaneous requests from the *same* user do. That is the whole design: lock per user, not per limiter.

Why `AtomicBoolean allowed` ?

The lambda passed to `compute` must return the new map value, so the "was it allowed?" answer has to leave some other way. A local `boolean` can't be assigned inside a lambda (it must be effectively final). The video shows the compiler error at 24:13 and offers two fixes: an `AtomicBoolean` or a one-element array. Here it is about mutability, not atomicity.

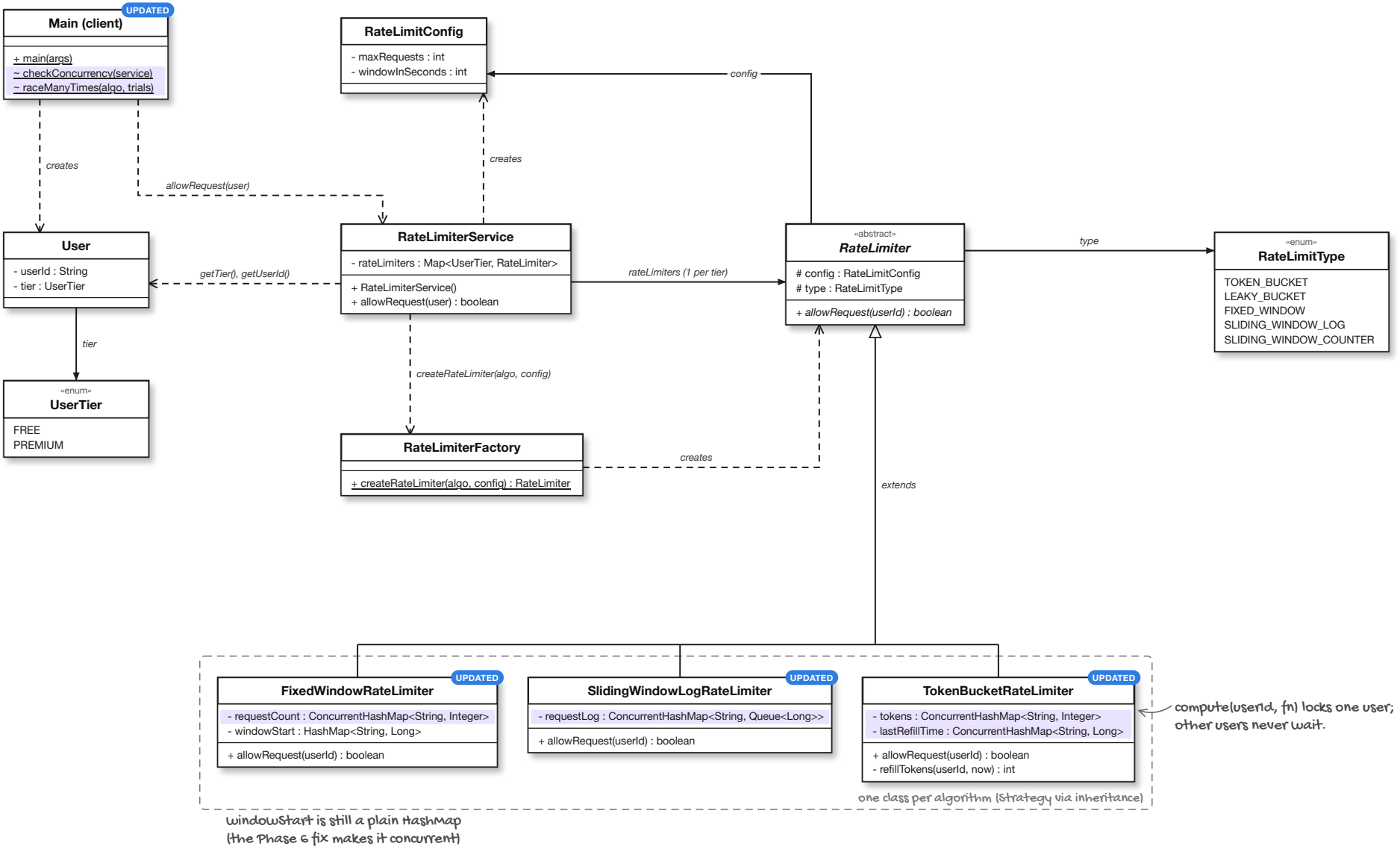
Rules for the function you pass to compute

Keep it short, because it holds the lock. Never update *other keys of the same map* inside it. Reading the same map, as `refillTokens` does with `tokens.getOrDefault`, is allowed. Writing to a *different* map is fine only if that map is concurrent too.

What the Phase 4 limiters do under the same test

Copying Phase 4's single-threaded limiters into this phase and running its `Main` three times gave this. Every run: the video's check allowed **13 of 20**. Of 500 races, the token bucket allowed exactly 10 in only 492–494 and the fixed window in 496–497. Two runs died with a `ConcurrentModificationException` from the sliding log's `ArrayDeque` and left the JVM hanging on the worker threads. Unsafe code fails rarely and differently every time, which is why one passing run proves nothing.

Phase 5 UML: same classes, safe state



No new classes. The shaded fields now hold `ConcurrentHashMap`s and every `allowRequest` runs inside `compute`. Note `windowStart`: the reference leaves it a plain `HashMap`.

The purple bars show exactly what thread safety changed relative to Phase 4. The algorithms are identical; only *where* they run changed: inside `compute`.

limiter/TokenBucketRateLimiter.java — CHANGED

```java
public class TokenBucketRateLimiter extends RateLimiter {
    private final Map<String, Integer> tokens = new ConcurrentHashMap<>();
    private final Map<String, Long> lastRefillTime = new ConcurrentHashMap<>();

    public TokenBucketRateLimiter(RateLimitConfig config) {
        super(config, RateLimitType.TOKEN_BUCKET);
    }

    @Override
    public boolean allowRequest(String userId) {
        AtomicBoolean allowed = new AtomicBoolean(false);
        long now = System.currentTimeMillis();

        tokens.compute(userId, (id, availableTokens) -> {
            int currentTokens = refillTokens(userId, now);

            if (currentTokens > 0) {
                allowed.set(true);           // mark allowed BEFORE we decrement
                return currentTokens - 1;    // consume 1 token
            } else {
                return currentTokens;        // remain at 0
            }
        });

        return allowed.get();
    }

    // free user refill rate = 60 / 10 = 6
    private int refillTokens(String userId, long now) {
        double refillRate = (double) config.getWindowInSeconds() / config.getMaxRequests();

        lastRefillTime.putIfAbsent(userId, now);
        long lastRefill = lastRefillTime.get(userId);

        long elapsedSeconds = (now - lastRefill) / 1000;
        int refillTokens = (int) (elapsedSeconds / refillRate);
        int currentTokens = tokens.getOrDefault(userId, config.getMaxRequests());
        currentTokens = Math.min(config.getMaxRequests(), currentTokens + refillTokens);

        if (refillTokens > 0) lastRefillTime.put(userId, now);
        return currentTokens;
    }
}
```

limiter/SlidingWindowLogRateLimiter.java CHANGED

```java
public class SlidingWindowLogRateLimiter extends RateLimiter {
    private final Map<String, Queue<Long>> requestLog = new ConcurrentHashMap<>();

    public SlidingWindowLogRateLimiter(RateLimitConfig config) {
        super(config, RateLimitType.SLIDING_WINDOW_LOG);
    }

    @Override
    public boolean allowRequest(String userId) {
        AtomicBoolean allowed = new AtomicBoolean(false);
        long now = System.currentTimeMillis() / 1000;

        requestLog.compute(userId, (id, log) -> {
            if (log == null) log = new ArrayDeque<>();

            while (!log.isEmpty() && (now - log.peek()) >= config.getWindowInSeconds()) {
                log.poll();
            }

            if (log.size() < config.getMaxRequests()) {
                log.add(now);       // record this request
                allowed.set(true); // mark allowed
            }

            return log;
        });

        return allowed.get();
    }
}
```

limiter/FixedWindowRateLimiter.java CHANGED

```java
public class FixedWindowRateLimiter extends RateLimiter {
    private final Map<String, Integer> requestCount = new ConcurrentHashMap<>();
    private final Map<String, Long> windowStart = new HashMap<>();

    public FixedWindowRateLimiter(RateLimitConfig config) {
        super(config, RateLimitType.FIXED_WINDOW);
    }

    @Override
    public boolean allowRequest(String userId) {
        AtomicBoolean allowed = new AtomicBoolean(false);

        long currentReqWindow = System.currentTimeMillis() / 1000 / config.getWindowInSeconds();

        requestCount.compute(userId, (id, count) -> {
            long lastReqWindow = windowStart.getOrDefault(id, currentReqWindow);

            if (lastReqWindow != currentReqWindow) {
                // window expired -> reset counter and window of last req
                windowStart.put(id, currentReqWindow);
                allowed.set(true);
                return 1; // first request in new window
            }

            if (count == null) count = 0;

            if (count < config.getMaxRequests()) {
                allowed.set(true);
                return count + 1; // increment count
            }

            return count; // remain same
        });

        return allowed.get();
    }
}
```

Main.java (the video's check, and 500 races per algorithm) CHANGED

```java
    // call allowRequest 20 times simultaneously (the video's check)
    static void checkConcurrency(RateLimiterService rateLimiterService) throws InterruptedException {
        User freeUser1 = new User("user1", UserTier.FREE);

        int threads = 20; // simulate 20 concurrent requests
        ExecutorService executor = Executors.newFixedThreadPool(threads);

        CyclicBarrier barrier = new CyclicBarrier(threads);
        CountDownLatch latch = new CountDownLatch(threads);
        AtomicInteger allowedCount = new AtomicInteger();

        for (int i = 1; i <= threads; i++) {
            executor.submit(() -> {
                try {
                    // all threads wait here until barrier is full
                    barrier.await();
                } catch (Exception e) {
                    e.printStackTrace();
                }

                if (rateLimiterService.allowRequest(freeUser1)) {
                    allowedCount.incrementAndGet();
                }

                latch.countDown();
            });
        }

        latch.await(); // wait for all threads to finish
        executor.shutdown();

        System.out.println(threads + " simultaneous requests for user1 (FREE, 10 req / 60 s): "
                + allowedCount.get() + " ALLOWED, " + (threads - allowedCount.get()) + " BLOCKED");
    }

    // Runs the same race many times on a fresh limiter. A thread-safe limiter allows exactly 10 every time.
    static void raceManyTimes(RateLimitType algo, int trials) throws Exception {
        int threads = 20;
        ExecutorService executor = Executors.newFixedThreadPool(threads);
        int exact = 0;

        for (int t = 0; t < trials; t++) {
            RateLimiter limiter = RateLimiterFactory.createRateLimiter(algo, new RateLimitConfig(10, 60));
            CyclicBarrier barrier = new CyclicBarrier(threads);
            List<Future<Boolean>> answers = new ArrayList<>();
            for (int i = 0; i < threads; i++) {
                answers.add(executor.submit(() -> {
                    barrier.await();
                    return limiter.allowRequest("user1");
                }));
            }

            int allowed = 0;
            for (Future<Boolean> answer : answers) {
                if (answer.get()) allowed++;
            }
            if (allowed == 10) exact++;
        }

        executor.shutdown();
        System.out.printf("%-19s %d/%d races allowed exactly 10 of 20%n", algo, exact, trials);
```

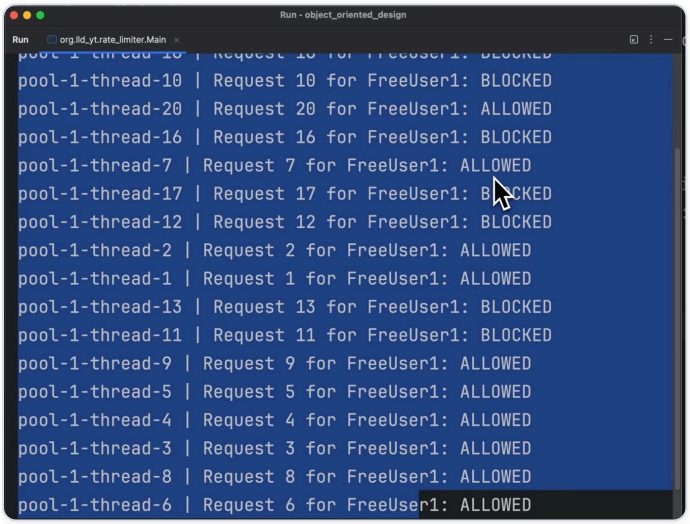
```
▶ bash Phase-5-Thread-Safety/run.sh

=== The video's check: 20 requests at the same instant ===
20 simultaneous requests for user1 (FREE, 10 req / 60 s): 10 ALLOWED, 10 BLOCKED

=== 500 races per algorithm (20 threads, limit 10) ===
TOKEN_BUCKET        500/500 races allowed exactly 10 of 20
FIXED_WINDOW        500/500 races allowed exactly 10 of 20
SLIDING_WINDOW_LOG  500/500 races allowed exactly 10 of 20
```

How the test makes a real race

- `CyclicBarrier(20)`: each thread waits at `barrier.await()` until all 20 are there, then all are released together.
- `CountDownLatch(20)`: `main` waits until every thread has counted down (the video's version). `raceManyTimes` waits on `Future.get()` instead.
- One race could pass by luck, so `raceManyTimes` repeats it 500 times on a fresh limiter. Correct code gets exactly 10 every time.



Video @ 31:25 · the video prints one line per thread, in a different order every run, and counts 10 ALLOWED by hand.

Checkpoint

1) Why is a `synchronized allowRequest` correct but a poor fit for requirement R4?
2) Two threads, two *different* users: do their `compute` calls wait for each other?
3) `windowStart` is a plain `HashMap` written inside `compute`. Why is that unsafe for two different users? Why doesn't it bite here? *(Phase 6.)*

PHASE
6

Extensions

Inject the clock · Fixed-window bug · Refill drift · 4th algorithm · Validation

Beyond the video: make time controllable so time-based behaviour can be tested exactly, use that to expose and fix two bugs in the reference code, build the sliding window counter from the requirements, and reject bad configuration.

Phase-6-Extensions/

`ManualClock` `SlidingWindowCounterRateLimiter` `Clock injection` `validation`

1. Inject the clock

Every limiter used to call `System.currentTimeMillis()`, so testing a refill meant sleeping, and a window boundary could land in the middle of any run. Now `RateLimiter` holds a `java.time.Clock`. Production code passes `Clock.systemUTC()` (the old one-argument constructors still do); tests pass a `ManualClock` and call `advanceSeconds(60)`. The factory and the service gain overloads that take a clock, so the video's demo can run on a frozen clock too.

clock/ManualClock.java — NEW

```java
public class ManualClock extends Clock {
    private volatile long millis;

    public ManualClock(long startMillis) {
        this.millis = startMillis;
    }

    public void advanceSeconds(long seconds) {
        millis += seconds * 1000;
    }

    @Override
    public long millis() {
        return millis;
    }

    @Override
    public Instant instant() {
        return Instant.ofEpochMilli(millis);
    }

    @Override
    public ZoneId getZone() {
        return ZoneOffset.UTC;
    }

    @Override
    public Clock withZone(ZoneId zone) {
        return this;
    }
}
```

limiter/RateLimiter.java — CHANGED

```java
public abstract class RateLimiter {
    protected final RateLimitConfig config;
    protected final RateLimitType type;
    protected final Clock clock;

    public RateLimiter(RateLimitConfig config,
     RateLimitType type, Clock clock) {
        this.config = config;
        this.type = type;
        this.clock = clock;
    }

    public abstract boolean allowRequest(String userId);
}
```

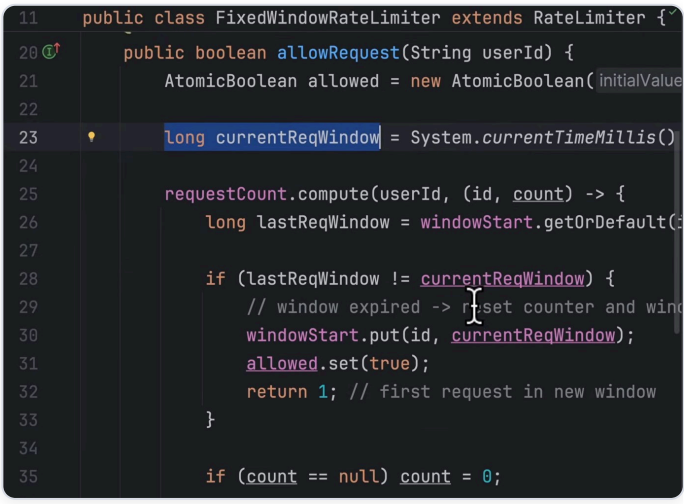
Why `java.time.Clock` and not a home-made interface?

It is the JDK's own abstraction for "what time is it", so production code needs no adapter: `Clock.systemUTC()` is the real clock. A test clock only has to subclass it and answer `millis()`, plus three one-line methods `Clock` requires.

2. Bug: the fixed window never resets

For a new user, `windowStart.getOrDefault(id, currentReqWindow)` returns the *current* window, so the "window changed" branch is skipped and `windowStart.put` never runs. The map stays empty forever, so every later window also looks "unchanged". Each user gets `maxRequests` in total, for good.

Running the Phase 5 classes on a fake clock confirms it: after 10 requests at t=0, a request at t=60 s and one at t=3600 s are both BLOCKED. The fix is a `-1` default, so a user's first request takes the reset branch and records its window. `windowStart` also becomes a `ConcurrentHashMap`, because now it really is written, for different users at once.



Video @ 27:35 · the line with the bug: getOrDefault(id, currentReqWindow) on line 26.

FixedWindowRateLimiter.java (allowRequest) CHANGED

```java
public boolean allowRequest(String userId) {
    AtomicBoolean allowed = new AtomicBoolean(false);

    long currentReqWindow = clock.millis() / 1000 / config.getWindowInSeconds();

    requestCount.compute(userId, (id, count) -> {
        long lastReqWindow = windowStart.getOrDefault(id, -1L); // -1 = no request yet

        if (lastReqWindow != currentReqWindow) {
            // first request, or window expired -> reset counter and window of last req
            windowStart.put(id, currentReqWindow);
            allowed.set(true);
            return 1; // first request in new window
        }

        if (count < config.getMaxRequests()) {
            allowed.set(true);
            return count + 1; // increment count
        }

        return count; // remain same
    });

    return allowed.get();
}
```

3. Bug: the token bucket loses partial refill time

The reference sets `lastRefillTime = now` whenever it adds tokens. With one token every 6 s, a request at t=11 adds 1 token (earned at t=6) and restarts the clock at 11. The 5 s already earned towards the next token are lost, so a request at t=12 is BLOCKED although a second token was due at t=12. Run on a fake clock, the Phase 5 class does exactly that. The fix advances `lastRefillTime` by the time the added tokens took; when the bucket is full it restarts from `now`, because a full bucket earns nothing. It also counts in milliseconds instead of whole seconds.

TokenBucketRateLimiter.java (refillTokens) — CHANGED

```
    // free user: one token every 60 / 10 = 6 s
    private int refillTokens(String userId, long now) {
        double refillIntervalMillis = config.getWindowInSeconds() * 1000.0 / config.getMaxRequests();

        lastRefillTime.putIfAbsent(userId, now);
        long lastRefill = lastRefillTime.get(userId);

        int refillTokens = (int) ((now - lastRefill) / refillIntervalMillis);
        int currentTokens = tokens.getOrDefault(userId, config.getMaxRequests());

        if (currentTokens + refillTokens >= config.getMaxRequests()) {
            // bucket is full: time spent full earns nothing, so the next token is a full interval away
            lastRefillTime.put(userId, now);
            return config.getMaxRequests();
        }

        if (refillTokens > 0) {
            // move on only by the time those tokens took, keeping progress towards the next one
            lastRefillTime.put(userId, lastRefill + (long) (refillTokens * refillIntervalMillis));
        }
        return currentTokens + refillTokens;
    }
```

4. The fourth algorithm: one class, one factory case

The sliding window counter keeps a small `Counter` per user (window index, current count, previous count). When a new window starts, current becomes previous, or 0 if a whole window passed with no requests. The `Counter` is mutable, which is safe only because it is touched solely inside `compute`.

SlidingWindowCounterRateLimiter.java (allowRequest) — NEW

```
    public boolean allowRequest(String userId) {
        AtomicBoolean allowed = new AtomicBoolean(false);
        long windowMillis = config.getWindowInSeconds() * 1000L;
        long now = clock.millis();
        long currentWindow = now / windowMillis;
        double elapsedShare = (double) (now % windowMillis) / windowMillis; // how far into the current window

        counters.compute(userId, (id, counter) -> {
            if (counter == null) counter = new Counter(currentWindow);

            if (counter.window != currentWindow) {
                // slide: current becomes previous (or 0 if a whole window passed with no requests)
                counter.previousCount = (counter.window == currentWindow - 1) ? counter.currentCount : 0;
                counter.currentCount = 0;
                counter.window = currentWindow;
            }

            double estimate = counter.previousCount * (1 - elapsedShare) + counter.currentCount;
            if (estimate < config.getMaxRequests()) {
                counter.currentCount++;
                allowed.set(true);
            }

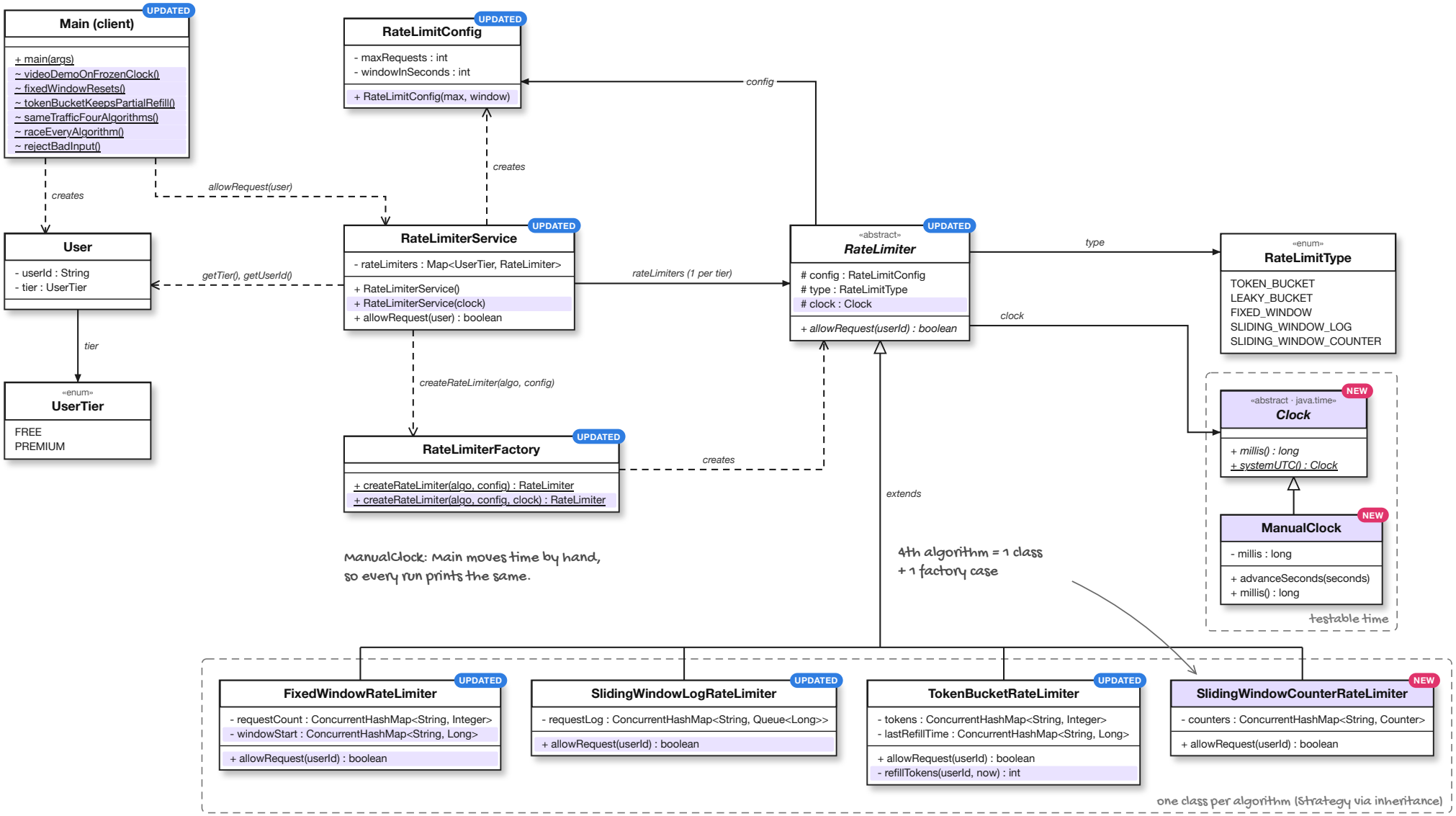
            return counter;
        });

        return allowed.get();
    }
```

RateLimiterFactory.java (the clock overload, with the new case) — CHANGED

```
    public static RateLimiter createRateLimiter(RateLimitType algo, RateLimitConfig config, Clock clock) {
        return switch (algo) {
            case TOKEN_BUCKET -> new TokenBucketRateLimiter(config, clock);
            case FIXED_WINDOW -> new FixedWindowRateLimiter(config, clock);
            case SLIDING_WINDOW_LOG -> new SlidingWindowLogRateLimiter(config, clock);
            case SLIDING_WINDOW_COUNTER -> new SlidingWindowCounterRateLimiter(config, clock);
            default -> throw new IllegalArgumentException("Unknown algorithm: " + algo);
        };
    }
```

Phase 6 UML: the final design



New: `SlidingWindowCounterRateLimiter`, `ManualClock` and the JDK's `Clock`. Shaded rows are the only members that changed; no class was restructured.

Main.java (the same traffic through all four algorithms) CHANGED

```java
    static void sameTrafficFourAlgorithms() {
        ManualClock clock = new ManualClock(0);
        List<RateLimiter> limiters = new ArrayList<>();
        for (RateLimitType algo : ALGORITHMS) {
            limiters.add(RateLimiterFactory.createRateLimiter(algo, new RateLimitConfig(10, 60), clock));
        }

        System.out.println("\n=== Same traffic, four algorithms (10 per 60 s): allowed per burst of 10 ===");
        System.out.printf("%-6s %-13s %-19s %-13s %s%n", "time", ALGORITHMS[0], ALGORITHMS[1], ALGORITHMS[2],
     ALGORITHMS[3]);
        int[] totals = new int[ALGORITHMS.length];
        long now = 0;
        for (long t : new long[]{50, 60, 90, 120}) {
            clock.advanceSeconds(t - now);
            now = t;
            int[] allowed = new int[ALGORITHMS.length];
            for (int a = 0; a < ALGORITHMS.length; a++) {
                for (int i = 0; i < 10; i++) {
                    if (limiters.get(a).allowRequest("user1")) allowed[a]++;
                }
                totals[a] += allowed[a];
            }
            System.out.printf("%-6s %-13d %-19d %-13d %d%n", "t=" + t, allowed[0], allowed[1], allowed[2], allowed[3]);
        }
        System.out.printf("%-6s %-13d %-19d %-13d %d%n", "total", totals[0], totals[1], totals[2], totals[3]);
    }
```

```
▶ bash Phase-6-Extensions/run.sh

=== The video's demo on a frozen clock ===
free user,     15 requests: 1-10 ALLOWED, 11-15 BLOCKED
premium user, 120 requests: 1-100 ALLOWED, 101-120 BLOCKED

=== Fix 1: the fixed window resets (10 per 60 s) ===
t=0   12 requests: 1-10 ALLOWED, 11-12 BLOCKED
t=60   1 request:  1 ALLOWED   (Phase 5 code: BLOCKED forever)

=== Fix 2: token bucket refill keeps partial progress (1 token / 6 s) ===
t=0   10 requests: 1-10 ALLOWED   (bucket now empty)
t=11   1 request:  1 ALLOWED   (token earned at t=6)
t=12   1 request:  1 ALLOWED   (token earned at t=12; Phase 5 code: BLOCKED)

=== Same traffic, four algorithms (10 per 60 s): allowed per burst of 10 ===
time   FIXED_WINDOW  SLIDING_WINDOW_LOG  TOKEN_BUCKET  SLIDING_WINDOW_COUNTER
t=50   10            10                  10            10
t=60   10            0                   1             0
t=90   0             0                   5             5
t=120  10            10                  5             5
total  30            20                  21            20

=== 500 races per algorithm (20 threads, limit 10) ===
FIXED_WINDOW            500/500 races allowed exactly 10 of 20
SLIDING_WINDOW_LOG      500/500 races allowed exactly 10 of 20
TOKEN_BUCKET            500/500 races allowed exactly 10 of 20
SLIDING_WINDOW_COUNTER  500/500 races allowed exactly 10 of 20

=== Bad input is rejected ===
createRateLimiter(LEAKY_BUCKET, ...) → Unknown algorithm: LEAKY_BUCKET
new RateLimitConfig(0, 60) → maxRequests must be positive, got 0
new RateLimitConfig(10, 0) → windowInSeconds must be positive, got 0
```

5. Reject bad configuration

A limit of 0 or a window of 0 makes no sense, and in the token bucket it would divide by zero. `RateLimitConfig` now refuses both in its constructor, so a bad config fails when it is created, with a clear message, not on the first request. The last lines of the output show it.

model/RateLimitConfig.java (constructor) — CHANGED

```
    public RateLimitConfig(int maxRequests, int windowInSeconds) {
        if (maxRequests ⩽ 0) {
            throw new IllegalArgumentException("maxRequests must be positive, got " + maxRequests);
        }
        if (windowInSeconds ⩽ 0) {
            throw new IllegalArgumentException("windowInSeconds must be positive, got " + windowInSeconds);
        }
        this.maxRequests = maxRequests;
        this.windowInSeconds = windowInSeconds;
    }
```

Reading the comparison table

Every algorithm gets the same traffic: 10 requests at t = 50, 60, 90 and 120 s, with a limit of 10 per 60 s.

- **Fixed window** allows 20 between t=50 and t=60: the boundary burst, now measured. (With the Phase 5 bug it would allow 10 in total and then nothing.)
- **Sliding log** is exact: nothing more until the t=50 requests are 60 s old, at t=110.
- **Token bucket** refills one token every 6 s: 1 token by t=60, 5 more by t=90 and by t=120.
- **Sliding counter** at t=90 still counts half of window 0 (10 × 0.5 = 5), so 5 more fit; at t=120 window 1's 5 requests count in full.

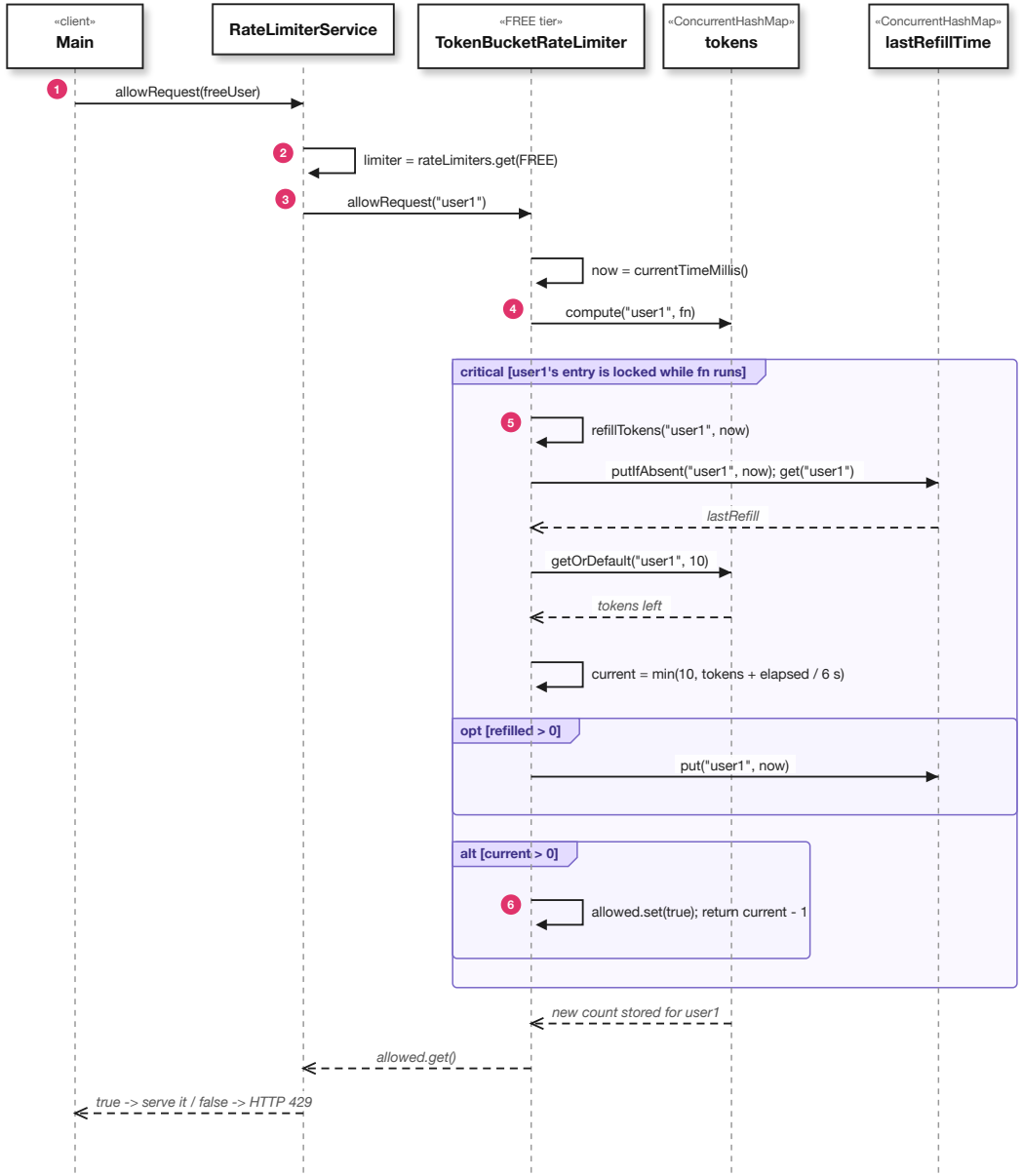
Why the races still pass with a frozen clock

The clock is shared and never moves during a race, so the only thing that varies is thread timing. 500/500 for the new `SlidingWindowCounterRateLimiter` shows its `Counter` object is safe: it is only touched inside `compute`, under that user's lock.

Checkpoint

1) Add LEAKY_BUCKET. List every file you touch.
2) Why does `ManualClock` extend `Clock` rather than being a separate interface the limiters know about?
3) In the token bucket, why reset `lastRefillTime` to `now` when the bucket is full?
4) Where would you reject a `null` user, and why there?

Putting it together: the complete code flow



Flow 1: `allowRequest` for a free user

1 The client only knows the user and calls the service.

2 The service looks up the limiter for the user's tier. FREE gives the token bucket.

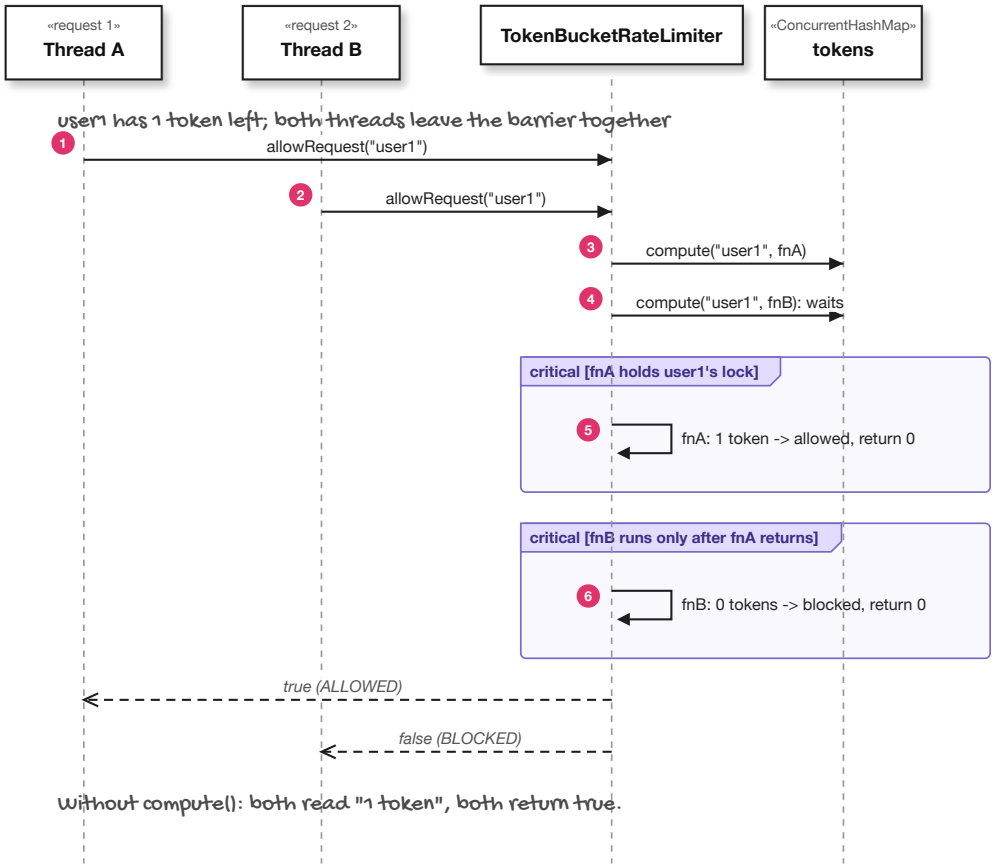
3 It delegates with just the user id. The limiter reads the time once, before taking the lock.

4 `tokens.compute("user1", fn)` locks user1's entry. Everything in the frame runs while that lock is held.

5 `refillTokens` reads the last refill time and the current tokens, adds what has been earned (capped at 10), and moves the refill time on if anything was added.

6 If a token is left, it is taken and `allowed` is set. The lambda's return value becomes the stored token count.

The answer travels back as a boolean. An API layer turns `false` into **HTTP 429**, ideally with a `Retry-After` header (11:17).



Flow 2: two requests race for the last token

1–2 Threads A and B call `allowRequest("user1")` at the same instant; user1 has one token left.

3–4 Both reach `compute`. A gets the lock on user1's entry; B's `compute` waits.

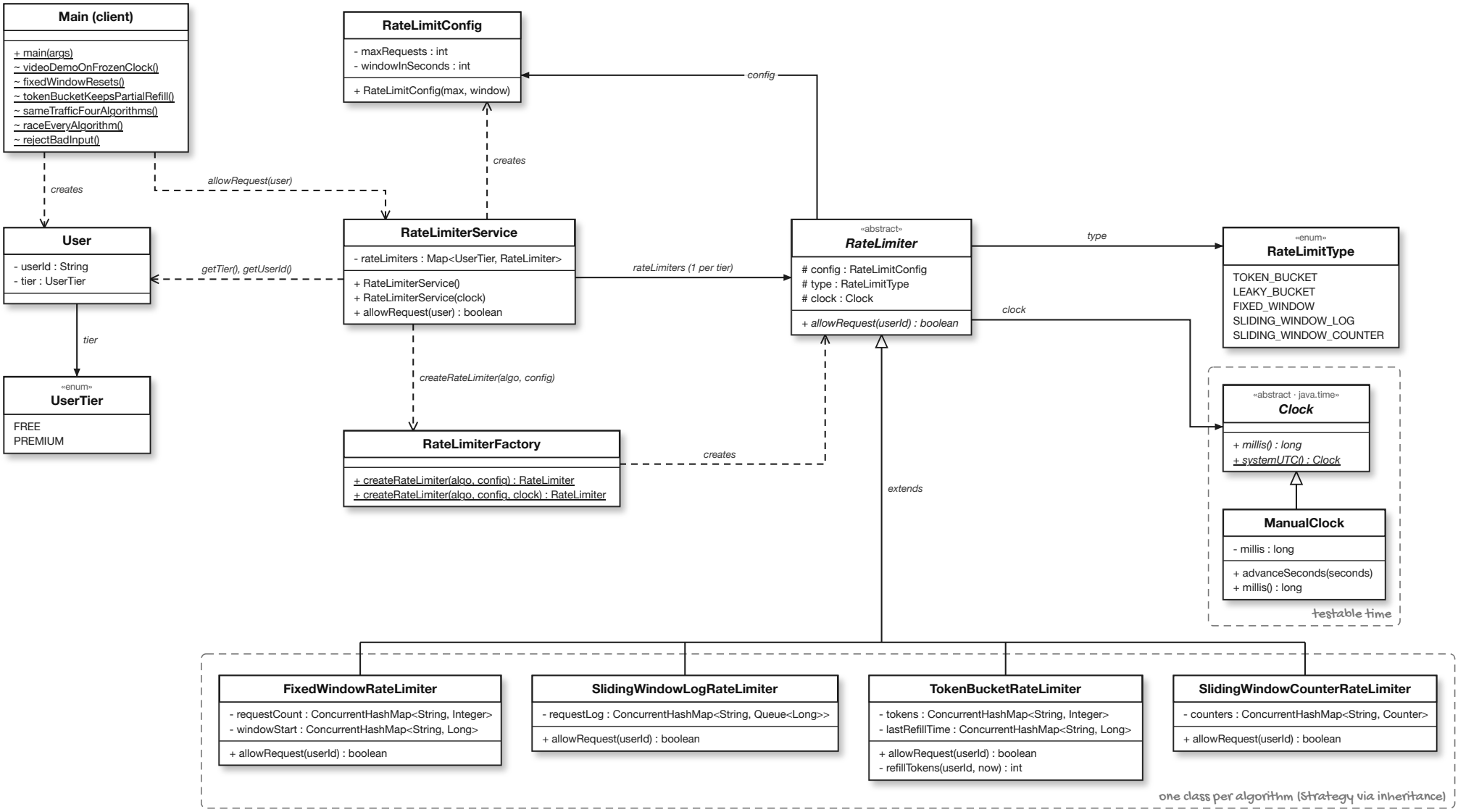
5 A's function sees 1 token, allows, and stores 0.

6 Only then does B's function run. It sees the 0 that A stored and blocks.

A request for user2 would not have waited at all: its entry has its own lock.

> **Explain it in 30 seconds**
>
> "A request comes from a user with a tier. **RateLimiterService** keeps one limiter per tier and routes by tier. Each limiter is a subclass of an abstract **RateLimiter**, one per algorithm, built by a **factory** from a **RateLimitType**, so a new algorithm is one class plus one case. Inside a limiter, per-user state lives in a **ConcurrentHashMap** keyed by user id, and every check runs in **compute()**, which makes the read-modify-write atomic per user without blocking other users."

The complete design (all six phases stitched together)



Phases 1–5 are the video's design. Phase 6 adds `SlidingWindowCounterRateLimiter`, the `Clock` / `ManualClock` pair, the two bug fixes and config validation. The same diagram is in `diagrams/complete-class-diagram.svg`, which you can zoom freely.

Notes on the reference code

Phase 5 has the same 11 classes (counting `Main`) and the same algorithms as the reference repository; a `diff` of the limiters, factory and service shows only Lombok, the package name and a few stray comments. The deliberate differences:

Reference	In these notes	Why
Lombok `@Getter`, `@AllArgsConstructor`	Plain constructors and getters	Compiles with just `javac`; nothing hidden from the reader
Package `org.nailyourinterview.lld.rate_limiter`	`ratelimiter`	Shorter paths in the notes
Algorithms are thread-safe from the start	Phases 2–4 use single-threaded versions of the same algorithms; Phase 5 is the reference	So `diff -ru Phase-4… Phase-5…` shows exactly what thread safety adds
`checkConcurrency` prints one line per thread, in random order	Prints the counts, plus 500 repeated races per algorithm	Output is the same on every run
Video's code (20:05): `lastRefillTime = new HashMap<>()`	`ConcurrentHashMap`, as the repository has now	It is written inside `compute` for different users at the same time

Where the whiteboard and the code disagree (diagrams follow the code)

- At 10:43 the video says free users get a fixed window and premium users a token bucket. The code (19:04) does the opposite: FREE → token bucket, PREMIUM → fixed window.
- The requirements example is 2 requests per 10 s (free) and 10 per 10 s (premium); the code uses 10 per 60 s and 100 per 60 s.
- The whiteboard's `RateLimitType` has four values; the code adds LEAKY_BUCKET, which has no class. The whiteboard draws a `SlidingWindowCounter` class; the code has none, and its factory throws for it.

Issues in the reference, fixed in Phase 6

Issue	Effect	Fix
Fixed window: `windowStart` is never written for a new user	The count never resets: each user gets `maxRequests` in total	Default to `-1` so the first request records its window
Fixed window: `windowStart` is a plain `HashMap` written inside `compute`	Once the first bug is fixed, concurrent writes for different users can corrupt it	`ConcurrentHashMap`
Token bucket: `lastRefillTime = now` after a refill	Partial progress towards the next token is lost	Advance by the time the added tokens took
Token bucket and sliding log use whole seconds	A 0.6 s refill or a window edge is rounded	Milliseconds

Where to take it next

Many servers

Each server's in-memory maps only see its own traffic. Move the state to a shared store such as Redis: `INCR` + `EXPIRE` for a fixed window, a sorted set for the sliding log, a Lua script for the token bucket so the read-modify-write stays atomic.

Memory for idle users

The maps keep every user id forever. Evict entries idle for longer than a window, with a scheduled sweep or a cache with expire-after-access.

Leaky bucket

The fifth type is already in the enum. Requests join a queue that drains at a fixed rate, which smooths output instead of allowing bursts. One class and one factory case.

Tell the client

Return **429** with `Retry-After` and remaining-quota headers. For that, `allowRequest` would return a small result object instead of a boolean.

Per-endpoint limits

Key the routing map by (tier, endpoint), so `/interpret_solution` can be strict while `/check` polling is loose.

Live configuration

Load tier limits from config and swap a tier's limiter with an atomic reference when they change, without a restart.

Exercise

Implement `LeakyBucketRateLimiter` in a copy of Phase 6, add its case to the factory and its column to `sameTrafficFourAlgorithms`, and predict the column before you run it.

Self-test: interview questions

Cover the grey answers, say yours out loud, then check.

1. **Why does `RateLimiterService` hold one limiter per *tier* rather than per user?**
   Rules vary by tier; state varies by user. The per-user state lives inside each limiter, in maps keyed by user id.
2. **Draw the fixed-window boundary problem.**
   N requests just before a boundary and N just after are in different windows: 2N within a short span.
3. **What does the sliding window log store, and what does that cost?**
   A timestamp per allowed request in the last window: O(N) per user. 100 per minute × a million users is 100 million timestamps.
4. **Token bucket: what are capacity and refill rate, in terms of the config?**
   Capacity = maxRequests (the largest burst); refill = window / maxRequests seconds per token (the average rate).
5. **Give the sliding window counter formula.**
   previous count × (share of the previous window still inside the sliding window) + current count, compared with the limit.
6. **Why `compute()` instead of a `synchronized allowRequest`?**
   Both are correct, but synchronized makes every user wait for every other user. compute locks only that user's entry.
7. **Why is `allowed` an `AtomicBoolean`?**
   A lambda can only use effectively final locals, and the lambda must return the new map value. AtomicBoolean (or boolean[1]) is a mutable holder.
8. **How do you test that a token arrives 6 seconds later without sleeping?**
   Inject a Clock and use a manual one in the test; advance it by 6 seconds.
9. **What is the bug in the reference fixed window?**
   windowStart is never written for a user's first window (getOrDefault returns the current window), so the count never resets.
10. **How would you add a fifth algorithm?**
    One RateLimiter subclass, one enum value if it's missing, one factory case. Nothing else changes (Open/Closed).
11. **Your service runs on 20 servers. What changes?**
    Shared state in Redis (or similar), with atomic operations or a Lua script per check; or per-server limits of N / 20 if approximation is acceptable.
12. **What should the API return when a request is blocked?**
    HTTP 429 Too Many Requests, with Retry-After and remaining-quota headers.

Final drill: draw the complete diagram from a blank page, then build Phase 5 from an empty folder and make its 500 races pass. ✎